DEVOPS ENGINEER COURSE

Git for DevOps Engineers

Version control, collaboration and GitOps

Beginner | Intermediate | Advanced

Concepts · hands-on labs · troubleshooting · interview Q&A · cheat sheet

Contents

Part 1 — Foundations 5

1.1 Why Version Control Exists 5
1.2 Installing Git 6
1.3 Configuring Git 7
1.4 The Three Trees 10
1.5 What Lives in .git 12
1.6 Lab: Set Up a Professional Git Environment 13
1.7 Key Takeaways 14

Part 2 — Everyday Git: Recording and Inspecting Changes 15

2.1 Creating and Obtaining Repositories 15
2.2 Checking State: git status 16
2.3 Staging: git add 17
2.4 Committing 18
2.5 Ignoring Files: .gitignore 20
2.6 Inspecting Changes: git diff 21
2.7 Reading History: git log 22
2.8 Moving and Removing Files 25
2.9 Discarding and Undoing Small Things 25
2.10 Lab: Build a Clean History 25
2.11 Key Takeaways 26

Part 3 — Branching, Merging and Remotes 27

3.1 What a Branch Really Is 27
3.2 Working With Branches 27
3.3 Merging 29
3.4 Merge Conflicts 30
3.5 Remotes 32
3.6 The Fork Workflow 34
3.7 Lab: Branch, Conflict, Collaborate 35
3.8 Key Takeaways 36

Part 4 — Rewriting History, Undoing and Recovery 37

4.1 Rebase 37
4.2 Merge vs Rebase: The Trade-offs 40
4.3 Cherry-pick 41
4.4 Undoing Things: restore, reset, revert 42
4.5 The Reflog: Your Safety Net 43
4.6 Stash 44
4.7 Tags, Semantic Versioning and Releases 45
4.8 Lab: Rewrite, Break and Recover 46
4.9 Key Takeaways 47

Part 5 — Git Internals and Power Tools 49

5.1 The Object Database 49

5.2 Refs, Packfiles and Garbage Collection 51
5.3 git bisect: Binary Search for Bugs 53
5.4 git blame: Who Changed This Line and Why 54
5.5 Worktrees: Multiple Branches Checked Out at Once 55
5.6 Submodules vs Subtrees 55
5.7 Monorepos: Sparse Checkout and Partial Clone 57
5.8 Large Files: Git LFS 58
5.9 Performance for Big Repositories 59
5.10 Lab: Inside the Machine 60
5.11 Key Takeaways 61

Part 6 — Hooks, Signing and Team Workflows 62
6.1 Git Hooks 62
6.2 Signing Commits and Tags 65
6.3 Branching Strategies 66
6.4 Pull Requests and Code Review 68
6.5 Branch Protection and Rulesets 70
6.6 CODEOWNERS 70
6.7 Platform Specifics 71
6.8 Lab: Enforce Quality Before It Reaches Main 72
6.9 Key Takeaways 73

Part 7 — Git in CI/CD, GitOps and Secrets Security 74
7.1 Git Events as Pipeline Triggers 74
7.2 GitHub Actions 74
7.3 GitLab CI 77
7.4 CI Checkout Best Practices 79
7.5 GitOps 79
7.6 Secrets: Prevention 82
7.7 Secrets: Response and History Cleanup 83
7.8 Lab: Pipeline, GitOps and a Leaked Key 85
7.9 Key Takeaways 86

Part 8 — Troubleshooting Common Disasters 87
8.1 Detached HEAD 87
8.2 Committed to the Wrong Branch 88
8.3 Pushed a Secret 88
8.4 Someone Force-Pushed Over Work 89
8.5 Merge Gone Wrong 89
8.6 Lost Commits After Reset or Rebase 90
8.7 Lost a Stash 91
8.8 Accidentally Committed a Huge File 91
8.9 “Diverged” Branches and Rejected Pushes 91
8.10 Other Frequent Errors 92
8.11 Lab: Disaster Drills 92
8.12 Key Takeaways 93

Part 9 — Capstone, Interview Q&A and Cheat Sheet 94
9.1 Capstone Lab: A Production-Grade Team Workflow 94

9.2 Interview Questions and Answers 97
9.3 Command Cheat Sheet 101
9.4 Key Takeaways 105

Part 1 — Foundations

This course takes you from “I can commit and push” to someone who can design a branching strategy for a platform team, recover a repository after a force-push accident, wire Git into CI/CD pipelines and GitOps controllers, and answer the hard questions in a senior DevOps interview. Every Part builds on the previous one and ends with a hands-on Lab and a list of key takeaways.

The commands use current Git (2.40 and later). Where older tutorials use `git checkout` for everything, this course prefers the newer, single-purpose commands `git switch` (change branches) and `git restore` (restore files), and assumes the default branch is called `main`.

1.1 Why Version Control Exists

A version control system (VCS) records changes to a set of files over time so you can answer four questions at any moment:

1. **What** changed?
2. **Who** changed it?
3. **When** did it change?
4. **Why** did it change?

For a DevOps engineer those questions are not academic. When production breaks at 02:00, “what changed in the Terraform repo since yesterday’s deploy?” is often the fastest path to the root cause. When an auditor asks who approved a firewall rule, the answer lives in commit history and pull request reviews. When you need to roll back, version control is the undo button for your entire infrastructure.

1.1.1 Centralized vs distributed

Property	Centralized (SVN, Perforce, CVS)	Distributed (Git, Mercurial)
Where history lives	One central server	Every clone has full history
Work offline	Limited (no commits, no log)	Fully: commit, branch, log, diff
Branching cost	Often expensive (directory copies)	Cheap (a 41-byte file pointing at a commit)
Single point of failure	Server	None; any clone can restore the server
Typical workflow	Lock / checkout / commit to server	Commit locally, then share via push/pull

Git is distributed: when you `git clone`, you get the whole project history, not just the latest snapshot. Your laptop is a full-fledged repository. The “central” server (GitHub, GitLab, Bitbucket, Gitea) is central only by convention.

1.1.2 Snapshots, not diffs

Many older systems store a file and then a sequence of deltas. Git conceptually stores a **snapshot** of the entire project at each commit. If a file did not change, the new snapshot simply points to the same stored content as before. This design is why branching, switching and merging are fast: Git compares snapshots rather than replaying chains of patches.

> **Note:** Internally Git does compress similar objects into deltas inside packfiles (Part 5), but that is a storage optimization. The data model you reason about is always "each commit is a full snapshot".

1.1.3 Integrity by hashing

Every object Git stores is named by a cryptographic hash of its contents (SHA-1 by default, SHA-256 optionally). A commit's hash covers its tree, its parents, its author and its message. Change a single byte anywhere in history and every descendant hash changes. This gives you tamper evidence for free and is why "rewriting history" is a real, visible event in Git.

1.2 Installing Git

Check what you have first:

```
git --version
```

```
git version 2.47.1
```

Platform	Recommended install	Notes
macOS	`brew install git`	Apple's Xcode Command Line Tools ship an older Git; Homebrew keeps you current
Debian/Ubuntu	`sudo apt install git`	For the newest release use `sudo add-apt-repository ppa:git-core/ppa`
RHEL/Fedora	`sudo dnf install git`	
Alpine (containers)	`apk add --no-cache git`	Common in CI images
Windows	Git for Windows (winget install Git.Git)	Includes Git Bash and Git Credential Manager

> **Tip:** In CI images and Dockerfiles, pin Git through the base image rather than installing "latest" at build time. Reproducible builds matter more than shiny features.

1.3 Configuring Git

Git configuration lives at three levels. Later levels override earlier ones.

Level	Flag	File	Scope
System	`--system`	`/etc/gitconfig` (or `$(prefix)/etc/gitconfig`)	Every user on the machine
Global	`--global`	`~/.gitconfig` or `~/.config/git/config`	Your user account
Local	`--local` (default)	`.git/config` in the repo	One repository
Worktree	`--worktree`	`.git/config.worktree`	One worktree (needs `extensions.worktreeConfig`)

To see every value and where it comes from:

```
git config --list --show-origin
```

```
file:/opt/homebrew/etc/gitconfig    credential.helper=osxkeychain
file:/Users/dev/.gitconfig          user.name=Dana Ops
file:/Users/dev/.gitconfig          user.email=dana@example.com
file:.git/config                    core.repositoryformatversion=0
file:.git/config                    remote.origin.url=git@github.com:acme/infra.git
```

1.3.1 Identity

Every commit embeds an author name and email. Set them once:

```
git config --global user.name "Dana Ops"
git config --global user.email "dana@example.com"
```

Platforms use the email to link commits to your account. If you contribute to work and personal projects from one laptop, use **conditional includes** so the right identity is picked automatically by directory:

```
# ~/.gitconfig
[user]
    name = Dana Ops
    email = dana@personal.example
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

```
# ~/.gitconfig-work
[user]
```

```
    email = dana.ops@acme.example
    signingkey = ~/.ssh/id_ed25519_work.pub
```

> **Warning:** The trailing slash in `gitdir:~/work/` matters. It means "any repository under this directory". Without it, the pattern matches only that exact path.

1.3.2 Editor, default branch and sane defaults

```
git config --global core.editor "vim"            # or "code --wait", "nano"
git config --global init.defaultBranch main
git config --global pull.rebase true             # rebase local commits on pull instead of
merge bubbles
git config --global rebase.autoStash true        # stash dirty changes around a rebase
git config --global fetch.prune true             # delete remote-tracking branches that
vanished on the server
git config --global push.autoSetupRemote true    # first push creates upstream automatically
git config --global diff.algorithm histogram     # usually more readable diffs
git config --global merge.conflictStyle zdiff3   # show the common ancestor in conflicts
git config --global rerere.enabled true          # remember conflict resolutions
git config --global column.ui auto
git config --global branch.sort -committerdate   # most recent branches first
```

Why each one matters:

- `pull.rebase true` keeps your local history linear when you sync with the remote, avoiding meaningless "Merge branch 'main' of github.com:…" commits.
- `fetch.prune true` stops your `git branch -r` output from filling with branches deleted months ago.
- `push.autoSetupRemote true` (Git 2.37+) removes the classic "fatal: The current branch has no upstream branch" message on the first push.
- `merge.conflictStyle zdiff3` (Git 2.35+) adds the base version to conflict markers, which makes resolution far less guesswork (Part 3).
- `rerere` ("reuse recorded resolution") records how you resolved a conflict and replays it automatically if the same conflict reappears, which is common during long rebases.

1.3.3 Line endings

Mixed Windows and Unix teams fight over CRLF vs LF. Shell scripts with CRLF fail with `/bin/bash^M: bad interpreter`. The robust fix is a committed `.gitattributes` file rather than per-user settings:

```
# .gitattributes
* text=auto eol=lf
*.ps1 text eol=crlf
*.png binary
*.jar binary
```

On Windows machines, `git config --global core.autocrlf true` converts on checkout; on macOS and Linux use `input` or leave it unset and rely on `.gitattributes`.

1.3.4 Aliases

Aliases save keystrokes and encode team conventions.

```
git config --global alias.st "status -sb"
git config --global alias.co switch
git config --global alias.br "branch -vv"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.lg "log --graph --oneline --decorate --all"
git config --global alias.unstage "restore --staged"
git config --global alias.amend "commit --amend --no-edit"
git config --global alias.undo "reset --soft HEAD~1"
```

An alias starting with `!` runs a shell command, which lets you build small tools:

```
git config --global alias.cleanup '!git branch --merged main | grep -vE "^\*|main" | xargs -r
git branch -d'
```

> **Note:** `xargs -r` is a GNU extension. On macOS BSD xargs it is accepted and ignored in recent versions, but test before sharing aliases across platforms.

1.3.5 Global gitignore

Some files should never be committed in any repository: editor swap files, OS metadata, local env files. Put them in a global excludes file instead of polluting every project's `.gitignore`:

```
git config --global core.excludesFile ~/.config/git/ignore
```

```
# ~/.config/git/ignore
.DS_Store
Thumbs.db
*.swp
.idea/
.vscode/
*.log
.env
.direnv/
```

Project-specific rules (build outputs, `node_modules/`, `.terraform/`) still belong in the project's own `.gitignore` so every contributor gets them.

1.3.6 Authentication: SSH and credential helpers

Remotes are reached over HTTPS or SSH.

SSH is the usual choice for engineers:

```
ssh-keygen -t ed25519 -C "dana@example.com"
cat ~/.ssh/id_ed25519.pub      # paste into GitHub/GitLab settings
ssh -T git@github.com
```

```
Hi dana-ops! You've successfully authenticated, but GitHub does not provide shell access.
```

HTTPS needs a token (passwords are no longer accepted by the major platforms). A credential helper stores it so you are not prompted every time:

Helper	Platform	Storage
`osxkeychain`	macOS	Keychain (encrypted)
`manager` (Git Credential Manager)	Windows, macOS, Linux	OS secure store, supports OAuth and MFA
`libsecret`	Linux desktop	GNOME Keyring / KWallet
`cache --timeout=3600`	Any	In memory, expires
`store`	Any	**Plain text** in `~/.git-credentials`

```
git config --global credential.helper osxkeychain
```

> **Warning:** Avoid `credential.helper store` on shared machines and CI runners. It writes your token unencrypted to disk. In CI, inject short-lived tokens through environment variables or the platform's built-in token instead.

1.4 The Three Trees

The single most important mental model in Git is that your files live in three places at once.

```
  Working Directory          Index (Staging Area)          HEAD (last commit)
  -----------------          --------------------          ------------------
  files you edit     --add-->  proposed next commit  --commit-->  committed snapshot
                     <-restore--                     <-restore --staged--
```

Tree	What it is	Stored in
Working directory	Real files on disk you open in an editor	Your project folder

Tree	What it is	Stored in
Index (staging area)	A binary file listing exactly what the next commit will contain	`.git/index`
HEAD	A pointer to the current branch, which points to the latest commit	`.git/HEAD` and `.git/refs/heads/*`

1.4.1 Why a staging area?

Beginners often ask why Git doesn't just commit everything that changed. The index exists so you can craft commits deliberately. Suppose you fixed a bug and, while there, reformatted an unrelated file. With staging you can commit the bug fix alone, with a clear message, and the formatting separately. That makes the history reviewable, revertable and bisectable.

1.4.2 Watching the trees move

```
echo "v1" > app.conf
git add app.conf          # working dir -> index
git commit -m "Add app.conf"   # index -> new commit, HEAD moves
echo "v2" > app.conf      # working dir differs from index
git status -s
```

```
 M app.conf
```

The two-column short status tells you which tree differs:

Column 1 (index vs HEAD)	Column 2 (working dir vs index)	Meaning
` `	`M`	Modified, not staged
`M`	` `	Modified and staged
`M`	`M`	Staged, then modified again
`A`	` `	New file staged
`?`	`?`	Untracked
`D`	` `	Deletion staged

```
git add app.conf
echo "v3" > app.conf
git status -s
```

```
MM app.conf
```

Now the index holds `v2`, the working directory holds `v3`, and HEAD holds `v1`. Three different versions of one file, which is exactly what the "three trees" mean. `git diff` shows working dir vs index (v2 to v3); `git diff --staged` shows index vs HEAD (v1 to v2); `git diff HEAD` shows working dir vs HEAD (v1 to v3).

1.4.3 The commands that move content between trees

Command	Effect
`git add <file>`	Copy working dir version into the index
`git restore <file>`	Copy index version over working dir (discard unstaged edits)
`git restore --staged <file>`	Copy HEAD version into the index (unstage)
`git restore --source=HEAD~2 <file>`	Copy a file from any commit into the working dir
`git commit`	Turn the index into a new commit and advance the branch
`git reset --soft/--mixed/--hard <commit>`	Move the branch pointer and optionally reset index and working dir (Part 4)

> **Interview:** "What is the difference between `git diff` and `git diff --staged`?" The first compares the working directory to the index (what you have not staged yet). The second compares the index to HEAD (what will go into the next commit).

1.5 What Lives in .git

Run `git init` and look inside:

```
mkdir demo && cd demo && git init
ls -1 .git
```

```
HEAD
config
description
hooks
info
objects
refs
```

Path	Purpose
`HEAD`	Which branch you are on, e.g. `ref: refs/heads/main`
`config`	Repository-local configuration
`objects/`	The content-addressed database: blobs, trees, commits, tags

Path	Purpose
`refs/heads/`	Local branches (each a file containing a commit hash)
`refs/tags/`	Tags
`refs/remotes/`	Remote-tracking branches like `origin/main`
`hooks/`	Sample client-side hook scripts (Part 6)
`index`	The staging area (appears after first `git add`)
`logs/`	Reflogs: history of where each ref pointed (Part 4)

Deleting `.git` deletes the repository; the remaining files become ordinary files. Copying the project folder including `.git` copies the whole repository.

1.6 Lab: Set Up a Professional Git Environment

Goal: configure Git the way you would on a new work laptop and observe the three trees.

1. Verify your version is 2.40 or newer:

```
bash git --version
```

2. Set identity, editor and defaults:

```
bash git config --global user.name "Your Name" git config --global user.email "you@example.com" git config --global init.defaultBranch main git config --global core.editor "code --wait" git config --global pull.rebase true git config --global fetch.prune true git config --global push.autoSetupRemote true git config --global merge.conflictStyle zdiff3 git config --global rerere.enabled true
```

3. Create a global ignore file and register it:

```
bash mkdir -p ~/.config/git printf ".DS_Store\n*.swp\n.env\n.idea/\n.vscode/\n" > ~/.config/git/ignore git config --global core.excludesFile ~/.config/git/ignore
```

4. Add the `lg` and `st` aliases from section 1.3.4.

5. Create an SSH key, add it to your Git hosting account and test with `ssh -T git@github.com` (or `git@gitlab.com`).

6. Create a sandbox repository and walk the three trees:

```
bash mkdir -p /tmp/three-trees && cd /tmp/three-trees git init echo "v1" > app.conf && git add app.conf && git commit -m "Add app.conf" echo "v2" > app.conf && git add app.conf echo "v3" > app.conf git status -s git diff git diff --staged git diff HEAD
```

Confirm you can explain each diff output.

7. Unstage and discard:

```
bash git restore --staged app.conf # index back to v1 git restore app.conf # working dir back to v1 cat app.conf
```

8. Run `git config --list --show-origin` and verify every setting comes from where you expect.

1.7 Key Takeaways

- Git is distributed: every clone holds full history, and the server is central only by agreement.
- Git models each commit as a full snapshot, identified by a content hash that also protects integrity.
- Configuration has system, global and local levels; use `includeIf` to keep work and personal identities separate.
- A handful of global settings (`pull.rebase`, `fetch.prune`, `push.autoSetupRemote`, `zdiff3`, `rerere`) prevent common daily friction.
- Use a global excludes file for personal noise and `.gitattributes` for line-ending policy.
- Prefer SSH keys or a secure credential helper; never store tokens in plain text on shared machines.
- Master the three trees: working directory, index, HEAD. Almost every Git command is a way of moving content between them.

Part 2 — Everyday Git: Recording and Inspecting Changes

This Part covers the commands you will type dozens of times a day. The goal is not just to know them but to use them precisely: staging exactly the right lines, writing commits a reviewer can understand in ten seconds, and digging through history quickly when something breaks.

2.1 Creating and Obtaining Repositories

2.1.1 git init

```
mkdir platform-infra && cd platform-infra
git init
```

```
Initialized empty Git repository in /home/dana/platform-infra/.git/
```

Useful variants:

```
git init -b main                # set the initial branch explicitly
git init --bare infra.git       # a server-side repository with no working directory
git init --object-format=sha256  # experimental SHA-256 repository
```

A **bare** repository has no working directory; it contains only what would normally be inside `.git`. Servers host bare repositories because nobody edits files there, and pushing into a non-bare repository's checked-out branch would leave its working directory out of sync.

2.1.2 git clone

```
git clone git@github.com:acme/platform-infra.git
git clone https://github.com/acme/platform-infra.git infra   # into directory "infra"
```

```
Cloning into 'platform-infra'...
remote: Enumerating objects: 4821, done.
remote: Counting objects: 100% (812/812), done.
remote: Compressing objects: 100% (301/301), done.
remote: Total 4821 (delta 560), reused 642 (delta 487), pack-reused 4009
Receiving objects: 100% (4821/4821), 1.92 MiB | 6.10 MiB/s, done.
Resolving deltas: 100% (2907/2907), done.
```

Clone does several things: creates the directory, initializes `.git`, adds a remote called `origin`, fetches all objects, creates remote-tracking branches (`origin/main`, `origin/feature-x`…), and checks out the default branch as a local branch tracking `origin/main`.

Clone option	Use case
`--depth 1`	Shallow clone, only latest commit; fast CI checkouts
`--branch v2.3.0` / `-b`	Check out a specific branch or tag
`--single-branch`	Fetch only one branch's history
`--filter=blob:none`	Partial clone: download file contents on demand (Part 5)
`--recurse-submodules`	Also clone submodules
`--mirror`	Exact copy of all refs; used for migrations and backups

2.2 Checking State: git status

```
git status
```

```
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   terraform/vpc.tf

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   terraform/variables.tf

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    terraform/outputs.tf
```

Read it top to bottom: which branch, how it relates to its upstream, what is staged, what is modified but unstaged, and what Git does not track yet. The hints in parentheses are accurate and worth reading.

For a compact view, `git status -sb` (short + branch):

```
## main...origin/main [ahead 1]
M  terraform/vpc.tf
 M terraform/variables.tf
?? terraform/outputs.tf
```

2.3 Staging: git add

Command	Stages
`git add file.tf`	One file
`git add terraform/`	A directory recursively
`git add -A`	All changes everywhere in the repo (new, modified, deleted)
`git add .`	All changes in the current directory and below
`git add -u`	Modified and deleted tracked files only, no new files
`git add -p`	Interactively choose hunks
`git add -N file`	"Intent to add": track a new file without staging content, so it shows in `git diff`

2.3.1 Patch mode: the professional's add

`git add -p` walks through each changed hunk and asks what to do:

```
diff --git a/terraform/vpc.tf b/terraform/vpc.tf
@@ -12,7 +12,7 @@ resource "aws_vpc" "main" {
-  cidr_block = "10.0.0.0/16"
+  cidr_block = var.vpc_cidr
(1/2) Stage this hunk [y,n,q,a,d,s,e,?]?
```

Key	Action
`y` / `n`	Stage / skip this hunk
`s`	Split into smaller hunks
`e`	Edit the hunk manually
`q`	Quit
`a` / `d`	Stage / skip this and all remaining hunks in the file

Patch mode is how you turn a messy afternoon of edits into three clean, focused commits. It also forces you to review every line before it is committed, which catches stray debug prints and accidentally pasted credentials.

> **Tip:** The same `-p` flag works with `git restore -p`, `git stash -p` and `git reset -p`.

2.4 Committing

```
git commit                      # opens editor for a message
git commit -m "fix(vpc): parameterize CIDR block"
git commit -am "docs: update runbook"   # stage tracked modified files and commit (skips new
files)
git commit --amend              # replace the last commit (message and/or content)
git commit --amend --no-edit    # add staged changes to last commit, keep message
git commit --fixup=<sha>        # mark a commit to be squashed into <sha> later (Part 4)
git commit -S                   # sign the commit (Part 6)
git commit -v                   # show the diff in the editor while writing the message
```

> **Warning:** `--amend` creates a new commit with a new hash. If you have already pushed the original, amending rewrites shared history and you will need a force push. Amend freely on local, unpushed work.

2.4.1 Writing good commit messages

A commit message is documentation that is permanently attached to the exact change it describes. Six months from now, `git blame` will lead someone to your commit, and the message is all they will have.

The widely accepted format:

```
Short summary in imperative mood, about 50 chars

Body wrapped at 72 characters explaining WHY the change was made,
what problem it solves, and any trade-offs or side effects. The
diff already shows WHAT changed; the message must explain why.

Refs: OPS-1423
```

Rules of thumb:

1. Subject in the imperative: “Add”, “Fix”, “Remove”, not “Added” or “Fixes”. It should complete the sentence “If applied, this commit will …”.
2. Keep the subject around 50 characters, hard limit about 72, no trailing period.
3. Blank line between subject and body; many tools depend on it.
4. Explain the motivation, the alternatives rejected, and the impact.
5. One logical change per commit. If the message needs “and”, consider splitting.

2.4.2 Conventional Commits

Conventional Commits is a lightweight specification that makes commit messages machine-readable. It powers automated changelogs and semantic version bumps (Part 4).

```
<type>(<optional scope>): <description>

[optional body]

[optional footer(s)]
```

Type	Meaning	SemVer effect (typical)
`feat`	New feature	Minor
`fix`	Bug fix	Patch
`docs`	Documentation only	None
`refactor`	Code change that neither fixes a bug nor adds a feature	None
`perf`	Performance improvement	Patch
`test`	Adding or fixing tests	None
`build`	Build system or dependencies	None
`ci`	CI configuration	None
`chore`	Maintenance tasks	None
`revert`	Reverts a previous commit	Depends

A breaking change is marked with `!` after the type or a `BREAKING CHANGE:` footer, and triggers a major bump:

```
feat(api)!: drop support for v1 auth tokens

BREAKING CHANGE: clients must use OAuth2 bearer tokens. The legacy
X-Auth-Token header is no longer accepted.
```

Examples from a DevOps repository:

```
feat(helm): add PodDisruptionBudget to ingress chart
fix(terraform): pin aws provider to ~> 5.40 to avoid S3 ACL regression
ci: cache Go modules between pipeline runs
chore(deps): bump alpine base image to 3.20
```

> **Tip:** Enforce the format with a `commit-msg` hook (Part 6) using a tool like commitlint, and enforce it again in CI on the pull request title if you squash-merge.

2.4.3 Atomic commits

An atomic commit contains one complete, self-consistent change: the code, its tests and its docs together, and nothing unrelated. Atomic commits pay off later:

- `git revert` of one commit removes one feature or fix cleanly.
- `git bisect` (Part 5) pinpoints a single culprit instead of a 2,000-line grab bag.
- `git cherry-pick` of a fix to a release branch does not drag in unrelated work.
- Reviewers can reason about the change.

2.5 Ignoring Files: .gitignore

`.gitignore` tells Git which untracked files to leave alone. It does not affect files that are already tracked.

```
# .gitignore for an infrastructure repo

# Terraform
.terraform/
*.tfstate
*.tfstate.*
crash.log
*.tfvars
!example.tfvars

# Python
__pycache__/
*.pyc
.venv/

# Node
node_modules/
dist/

# Secrets and local config
.env
.env.*
!.env.example
*.pem
*.key

# Kubernetes local
kubeconfig*
```

2.5.1 Pattern syntax

Pattern	Matches
`*.log`	Any `.log` file in any directory

Pattern	Matches
`/build`	`build` only at the repo root
`build/`	Any directory named `build` (not a file)
`docs/**/*.pdf`	PDFs at any depth under `docs/`
`!keep.log`	Re-include a file excluded by an earlier pattern
`#`	Comment

> **Warning:** You cannot re-include a file if its parent directory is excluded. `logs/` followed by `!logs/keep.txt` does not work; use `logs/*` then `!logs/keep.txt`.

2.5.2 Where ignore rules live

Location	Shared with team?	Use for
`.gitignore` (any directory)	Yes, committed	Project build outputs, dependencies
`.git/info/exclude`	No	Personal, repo-specific ignores
`core.excludesFile` (global)	No	Editor and OS files everywhere

2.5.3 Debugging and fixing ignore rules

Why is a file ignored?

```
git check-ignore -v terraform/prod.tfvars
```

```
.gitignore:7:*.tfvars   terraform/prod.tfvars
```

Stop tracking a file that was committed before the ignore rule existed:

```
git rm --cached terraform/terraform.tfstate
git commit -m "chore: stop tracking terraform state"
```

`--cached` removes it from the index but leaves the file on disk. Note that the file is still present in history; if it held secrets, see Part 6.

2.6 Inspecting Changes: git diff

Command	Compares
`git diff`	Working dir vs index

Command	Compares
`git diff --staged` (alias `--cached`)	Index vs HEAD
`git diff HEAD`	Working dir vs HEAD
`git diff main feature`	Tip of main vs tip of feature
`git diff main...feature`	Merge base of the two vs feature (what the PR introduces)
`git diff HEAD~3 -- path/`	Limit to a path
`git diff --stat`	Summary of files and line counts
`git diff --name-only`	Just file names (great in CI scripts)
`git diff --word-diff`	Word-level changes, useful for docs and YAML
`git diff --color-moved`	Highlight moved blocks differently from edits

```
git diff --stat main...feature/pdb
```

```
 charts/ingress/templates/pdb.yaml | 14 ++++++++++++++
 charts/ingress/values.yaml        |  4 ++++
 2 files changed, 18 insertions(+)
```

> **Interview:** "What's the difference between `git diff A..B` and `git diff A...B`?" Two dots compares the two tips directly. Three dots compares the merge base of A and B against B, i.e. only the changes made on B since it diverged. Three dots is what a pull request shows.

In CI, `git diff --name-only` drives selective builds:

```
CHANGED=$(git diff --name-only origin/main...HEAD)
if echo "$CHANGED" | grep -q '^terraform/'; then
  terraform -chdir=terraform plan
fi
```

2.7 Reading History: git log

2.7.1 Formats

```
git log --oneline --graph --decorate -n 10
```

```
* 9f2c1ab (HEAD -> main, origin/main) feat(helm): add PDB to ingress chart
*   4e7d0c2 Merge pull request #88 from acme/fix/tf-provider
|\
```

```
| * b13a9e5 fix(terraform): pin aws provider to ~> 5.40
|/
* 70aa3d1 ci: cache Go modules
* 1c5e8f0 docs: add on-call runbook
```

```
git log --pretty=format:"%h %an %ar %s" -n 3
```

```
9f2c1ab Dana Ops 2 hours ago feat(helm): add PDB to ingress chart
4e7d0c2 Lee Chen 1 day ago Merge pull request #88 from acme/fix/tf-provider
b13a9e5 Lee Chen 1 day ago fix(terraform): pin aws provider to ~> 5.40
```

Common placeholders: `%H` full hash, `%h` short hash, `%an` author name, `%ae` author email, `%ad` author date, `%ar` relative date, `%cn` committer name, `%s` subject, `%b` body, `%d` ref names.

2.7.2 Filtering

Command	Shows
`git log -n 5`	Last 5 commits
`git log --since="2 weeks ago" --until=yesterday`	Date range
`git log --author="Lee"`	By author (regex)
`git log --grep="OPS-1423"`	Commit message contains text
`git log -S "db_password"`	“Pickaxe”: commits that add or remove that string
`git log -G "timeout\s*=\s*\d+"`	Commits whose diff matches a regex
`git log -- terraform/vpc.tf`	Commits touching a path
`git log --follow -- old/name.tf`	Follow a file across renames
`git log -p -- file`	Include the patch
`git log --stat`	Files changed per commit
`git log main..feature`	Commits on feature not yet in main
`git log --merges` / `--no-merges`	Only / exclude merge commits
`git log --first-parent main`	Mainline history only (one entry per merged PR)
`git log -L 20,40:app.py`	History of a line range
`git log -L :handler:app.py`	History of a function

The pickaxe (`-S`) deserves special mention for DevOps. “When was this config key introduced?” and “Did anyone ever commit an AWS key?” are both one command:

```
git log -S "AKIA" --all --oneline
```

2.7.3 Revision syntax

Git lets you name commits in many ways. Learn these and every command becomes more powerful.

Syntax	Meaning
`HEAD`	Current commit
`HEAD~1` , `HEAD~3`	First parent, three generations back
`HEAD^2`	Second parent of a merge commit
`main@{1}`	Where main pointed one move ago (reflog)
`main@{yesterday}`	Where main pointed yesterday (local reflog)
`@{u}` / `@{upstream}`	The upstream of the current branch
`v1.2.0^{commit}`	The commit a tag points to
`:/fix login`	Youngest commit whose message matches “fix login”
`A..B`	Commits reachable from B but not A
`A...B`	Commits reachable from either but not both (symmetric difference)

```
        HEAD~2  HEAD~1  HEAD
   o-------o-------o-------o   main
                    \
                     o--o      feature  (main..feature = these two)
```

2.7.4 git show

`git show` displays one object, typically a commit with its diff:

```
git show 9f2c1ab
git show HEAD~2:terraform/vpc.tf     # file content at that commit
git show v1.4.0 --stat
```

2.7.5 git shortlog

```
git shortlog -sn --since="3 months ago"
```

```
    42  Dana Ops
    31  Lee Chen
     9  dependabot[bot]
```

Good for release notes and understanding who knows a codebase.

2.8 Moving and Removing Files

```
git mv old.tf new.tf     # rename and stage
git rm obsolete.tf       # delete and stage
git rm --cached secret.env   # untrack but keep on disk
git rm -r --cached .terraform/
```

Git does not store renames explicitly. It detects them by comparing content similarity at diff time, which is why `git log --follow` and `git diff -M` work even if you used plain `mv` followed by `git add -A`.

2.9 Discarding and Undoing Small Things

Situation	Command
Throw away unstaged edits to a file	`git restore file`
Unstage a file but keep edits	`git restore --staged file`
Unstage and discard	`git restore --staged --worktree file`
Get a file back as it was 3 commits ago	`git restore --source=HEAD~3 file`
Fix the last commit message	`git commit --amend`
Forgot to add a file to last commit	`git add file && git commit --amend --no-edit`
Delete untracked files (preview first)	`git clean -n` then `git clean -fd`
Also delete ignored files	`git clean -fdx`

> **Warning:** `git restore file` and `git clean -f` destroy uncommitted work that Git has never stored. There is no reflog for it. Preview with `git clean -n` and consider `git stash -u` if unsure.

2.10 Lab: Build a Clean History

Goal: practise patch staging, Conventional Commits, ignore rules and history search.

1. Create a repository:

`bash mkdir -p /tmp/lab2 && cd /tmp/lab2 && git init`

2. Add a Terraform-like file and an ignore file:

`bash cat > main.tf <<'EOF' resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" } EOF printf ".terraform/\n*.tfstate\n*.tfvars\n!example.tfvars\n" > .gitignore git add .gitignore main.tf git commit -m "feat(vpc): add base VPC"`

3. Make two unrelated edits in one file: change the CIDR to `var.vpc_cidr`, and add a comment block at the bottom. Then stage only the CIDR change:

`bash git add -p main.tf # answer y for the CIDR hunk, n for the comment hunk git diff --staged git commit -m "refactor(vpc): parameterize CIDR block" git add main.tf git commit -m "docs(vpc): explain VPC sizing"`

If both edits end up in one hunk, use `s` to split or `e` to edit.

4. Prove the ignore rules work:

`bash touch prod.tfvars example.tfvars terraform.tfstate git status -s git check-ignore -v prod.tfvars`

Only `example.tfvars` should appear as untracked.

5. Simulate an accidental commit and untrack it:

`bash git add -f terraform.tfstate && git commit -m "oops: add state" git rm --cached terraform.tfstate git commit -m "chore: stop tracking terraform state"`

6. Search history:

`bash git log --oneline --graph git log -S "vpc_cidr" --oneline git log --stat -- terraform.tfstate git show HEAD~2:main.tf`

7. Write a commit with a body explaining why, using `git commit -v`, and view it with `git log -1`.

2.11 Key Takeaways

- `git clone` creates `origin`, remote-tracking branches and a local tracking branch; options like `--depth` and `--filter` control how much you download.
- Read `git status` carefully; `git status -sb` gives a dense summary.
- `git add -p` is the tool for building focused, reviewed commits.
- Write commit messages in the imperative, explain why, and adopt Conventional Commits to enable automation.
- `.gitignore` only affects untracked files; use `git rm --cached` to untrack and `git check-ignore -v` to debug.
- `git diff A...B` shows what a branch introduces; `git diff --name-only` drives selective CI.
- `git log` with `-S`, `--grep`, `--follow`, `-L` and revision ranges answers almost any “when and why did this change” question.
- `git restore` and `git clean` destroy uncommitted work irreversibly; preview first.

Part 3 — Branching, Merging and Remotes

Branches are what make Git a collaboration tool rather than a backup tool. This Part explains what a branch really is, how the different kinds of merge work, how to resolve conflicts with confidence, and how local branches relate to the branches on a remote server.

3.1 What a Branch Really Is

A branch is a movable pointer to a commit. Physically, it is a tiny file in `.git/refs/heads/` containing a 40-character hash (or an entry in `.git/packed-refs`).

```
cat .git/refs/heads/main
```

```
9f2c1ab4d1e8c3a7f6b0e2d9c5a4b3f2e1d0c9b8
```

`HEAD` is a pointer to a branch (normally) that tells Git "this is where the next commit goes":

```
cat .git/HEAD
```

```
ref: refs/heads/main
```

When you commit, Git creates the commit with the current tip as its parent, then moves the branch that HEAD points to forward to the new commit.

```
Before commit:                    After commit:

     HEAD -> main                        HEAD -> main
                |                                  |
  A <- B <- C                         A <- B <- C <- D
```

Because a branch is just a pointer, creating one is instantaneous and costs nothing, no matter how big the repository is. That is the main reason Git workflows lean so heavily on short-lived branches.

3.2 Working With Branches

```
git branch                      # list local branches
git branch -vv                  # with last commit and upstream tracking info
git branch -a                   # include remote-tracking branches
git branch feature/pdb          # create without switching
git switch feature/pdb          # switch to it
git switch -c feature/pdb       # create and switch in one step
```

```
git switch -c hotfix/cve origin/release/2.4   # create from a specific start point
git switch -                      # go back to the previous branch
git branch -m old-name new-name   # rename
git branch -d feature/pdb         # delete (refuses if unmerged)
git branch -D feature/pdb         # force delete
git branch --merged main          # branches already merged into main
git branch --no-merged main       # branches with work not in main
```

```
$ git branch -vv
  feature/pdb  3c1d2e9 [origin/feature/pdb: ahead 2] feat(helm): add PDB
* main         9f2c1ab [origin/main] ci: cache Go modules
  old-spike    a77e010 wip: try kustomize
```

> **Note:** `git checkout` still works for all of this, but it overloads two unrelated jobs (switching branches and overwriting files). `git switch` and `git restore`, introduced in Git 2.23, split those jobs so a typo cannot silently discard your edits.

3.2.1 Switching with uncommitted changes

If your uncommitted changes do not conflict with the target branch, Git carries them over. If they would be overwritten, Git refuses:

```
error: Your local changes to the following files would be overwritten by checkout:
    values.yaml
Please commit your changes or stash them before you switch branches.
```

Options: commit them, `git stash` them (Part 4), or `git switch --merge` to attempt a three-way merge of your local changes onto the new branch.

3.2.2 Branch naming conventions

Pick a convention and enforce it. A common one:

Prefix	Purpose	Example
`feature/` or `feat/`	New capability	`feature/OPS-812-pdb`
`fix/`	Bug fix	`fix/ingress-timeout`
`hotfix/`	Urgent production fix	`hotfix/cve-2024-3094`
`release/`	Release stabilization	`release/2.4`
`chore/`, `docs/`, `ci/`	Non-functional changes	`ci/cache-go-mod`

Prefixes let you apply branch protection rules and CI triggers by pattern (`release/*`), and including a ticket ID links work to your tracker automatically.

3.3 Merging

Merging combines the work of two branches. Git picks one of two shapes depending on the history.

3.3.1 Fast-forward merge

If the target branch has not moved since the feature branched off, there is nothing to combine. Git simply moves the pointer forward.

```
Before:                          After git merge feature (on main):

  A---B  main                      A---B---C---D  main, feature
       \
        C---D  feature
```

```
git switch main
git merge feature
```

```
Updating 9f2c1ab..3c1d2e9
Fast-forward
 charts/ingress/templates/pdb.yaml | 14 +++++++++++++
 1 file changed, 14 insertions(+)
```

No merge commit is created; history stays linear.

3.3.2 Three-way merge

If both branches have new commits, Git finds the **merge base** (the best common ancestor), compares both tips against it, and creates a new **merge commit** with two parents.

```
Before:                          After git merge feature (on main):

  A---B---E  main                  A---B---E-------M  main
       \                                \         /
        C---D  feature                   C---D----   feature
```

```
git merge-base main feature    # shows B
git merge feature
```

```
Merge made by the 'ort' strategy.
 charts/ingress/templates/pdb.yaml | 14 +++++++++++++
 1 file changed, 14 insertions(+)
```

It is called “three-way” because three versions are compared: base (B), ours (E) and theirs (D). For each change, if only one side touched it, that side wins automatically. Only when both sides changed the same region differently is there a conflict.

> **Note:** `ort` (“Ostensibly Recursive’s Twin”) has been the default merge strategy since Git 2.34. It replaced `recursive` and is significantly faster, especially with renames.

3.3.3 Controlling merge shape

Option	Effect	When to use
`--ff` (default)	Fast-forward if possible, otherwise merge commit	Default local behaviour
`--no-ff`	Always create a merge commit	Preserve the fact that a feature existed as a unit (GitFlow)
`--ff-only`	Refuse unless fast-forward is possible	Keeping `main` strictly linear; safe `pull`
`--squash`	Stage the combined changes but do not commit or record the merge	Collapse a branch into one commit

```
--no-ff result:                    --squash result:

  A---B----------M  main              A---B---S  main   (S = C+D as one commit,
       \        /                                       feature not recorded as merged)
        C-------D  feature
```

> **Warning:** After `git merge --squash`, Git does not know the feature branch was merged. `git branch -d feature` will complain it is unmerged; that is expected. Delete it with `-D` once you have verified the squash commit.

3.4 Merge Conflicts

A conflict means Git found two changes to the same lines (or a change versus a delete) and refuses to guess. Conflicts are normal; they are not errors.

3.4.1 Anatomy of a conflict

Suppose `main` set replicas to 3 and `feature` set them to 4, starting from 2:

```
git merge feature
```

```
Auto-merging values.yaml
CONFLICT (content): Merge conflict in values.yaml
Automatic merge failed; fix conflicts and then commit the result.
```

With `merge.conflictStyle=zdiff3` the file looks like this:

```
<<<<<<< HEAD
replicas: 3
||||||| 10a799a
replicas: 2
=======
replicas: 4
>>>>>>> feature
image: app:1.0
```

Section	Meaning
From `<<<<<<< HEAD` to the base marker (seven vertical bars)	Ours: the branch you are on
From the base marker to `=======`	Base: the common ancestor
Between `=======` and `>>>>>>>`	Theirs: the branch being merged

The base section is why `zdiff3` is worth enabling: you can see that the original was 2, that main raised it to 3, and feature raised it to 4. Without the base you only see “3 vs 4” and must guess what each side intended.

3.4.2 Resolution workflow

```
git status                      # lists "both modified" files
# edit the file, remove markers, keep the correct content
git add values.yaml             # mark as resolved
git merge --continue            # or: git commit
```

Helpful tools during a conflict:

Command	Purpose
`git diff`	Shows a combined diff of the conflict
`git log --merge -p`	Commits on either side that touched conflicted files
`git checkout --ours file` / `--theirs file`	Take one side wholesale
`git restore --source=feature file`	Take a file from a specific branch
`git mergetool`	Launch a configured visual tool (vimdiff, meld, VS Code)
`git merge --abort`	Give up and return to the pre-merge state

```
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait --merge $REMOTE $LOCAL $BASE $MERGED'
```

> **Warning:** During a **rebase**, “ours” and “theirs” are swapped compared to a merge. “Ours” is the branch you are rebasing onto (e.g. main), and “theirs” is your commit being replayed. This trips up even experienced engineers.

3.4.3 Reducing conflicts

- Keep branches short-lived and merge or rebase from main frequently.
- Avoid sweeping reformatting mixed with logic changes; land formatting in its own commit.
- Split large generated files or lock files, or mark them with merge drivers in `.gitattributes`.
- Enable `rerere` so a resolution done once is reused automatically.
- In config repositories (Helm values, Kustomize overlays), favour one file per environment or service so teams rarely edit the same lines.

3.5 Remotes

A **remote** is a named URL of another copy of the repository. `origin` is just the default name `git clone` gives the source.

```
git remote -v
```

```
origin  git@github.com:acme/platform-infra.git (fetch)
origin  git@github.com:acme/platform-infra.git (push)
```

```
git remote add upstream git@github.com:cncf/project.git
git remote rename origin fork
git remote set-url origin git@gitlab.com:acme/platform-infra.git
git remote remove old
git remote show origin          # detailed tracking info
```

3.5.1 Remote-tracking branches

`origin/main` is a **remote-tracking branch**: your local, read-only record of where `main` was on `origin` the last time you talked to it. It is updated only by `fetch` (and `pull`, which fetches). You never commit to it directly.

```
Local repository

  refs/heads/main            your branch            -> D
  refs/remotes/origin/main   last-known server main -> C
```

```
  A---B---C  origin/main
           \
            D  main          (git status: "ahead 1")
```

3.5.2 fetch, pull and push

Command	What it does
`git fetch`	Download new objects and update `origin/*`. Never touches your working files or local branches. Always safe.
`git fetch --all --prune`	Fetch every remote and delete stale remote-tracking branches
`git pull`	`git fetch` then integrate `@{u}` into the current branch (merge, or rebase if `pull.rebase=true`)
`git pull --ff-only`	Only update if a fast-forward is possible
`git push`	Upload commits and move the branch on the remote
`git push -u origin feature`	Push and set upstream tracking
`git push origin --delete feature`	Delete a remote branch
`git push --tags` / `git push origin v1.2.0`	Push tags
`git push --force-with-lease`	Overwrite remote branch only if it is still where you last saw it

> **Tip:** Many experienced engineers prefer `git fetch` followed by an explicit `git rebase origin/main` or `git merge origin/main` over `git pull`. It separates "see what changed" from "integrate it", giving you a chance to inspect with `git log HEAD..origin/main` first.

3.5.3 Upstream tracking

A local branch can have an **upstream**: the remote branch it syncs with by default. It is what makes plain `git pull`, `git push` and the "ahead/behind" message in `git status` work.

```
git branch --set-upstream-to=origin/main main
git branch -u origin/feature/pdb
git rev-list --left-right --count main...@{u}
```

```
1   3
```

That output means one commit ahead and three behind.

3.5.4 Why a push is rejected

```
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:acme/platform-infra.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally.
```

Git only accepts a push that fast-forwards the remote branch, so it can never silently drop someone else's commits. Fetch, integrate (rebase or merge), then push again.

3.5.5 Force pushing safely

When you have rewritten your own branch (rebase, amend), you must force push. Never use bare `--force` on shared branches.

```
git push --force-with-lease
git push --force-with-lease=feature/pdb:3c1d2e9   # expect remote to be exactly at this
commit
git push --force-if-includes --force-with-lease    # also require you've integrated the
remote tip
```

`--force-with-lease` checks that the remote branch still points to what your `origin/feature` says. If a teammate pushed in the meantime, the push is refused instead of erasing their work. Be aware that a background `git fetch` (some IDEs do this) updates `origin/*` and can defeat the lease; `--force-if-includes` (Git 2.30+) closes that gap by verifying the remote tip is actually in your local branch's history.

3.6 The Fork Workflow

Open source projects and some enterprises do not give contributors write access to the main repository. Contributors **fork** (a server-side clone under their own account), push to their fork and open a pull request against the original.

```
   upstream (acme/project)  <---- pull request ----+
         |                                         |
         | fork (server-side copy)                 |
         v                                         |
   origin (dana/project)  <---- git push ----  local clone
         ^                                         |
         +---------- git fetch upstream -----------+
```

```
git clone git@github.com:dana/project.git
cd project
git remote add upstream git@github.com:acme/project.git
git fetch upstream
git switch -c fix/typo upstream/main
# ...commit...
```

```
git push -u origin fix/typo
# open PR: dana:fix/typo -> acme:main

# keep your fork's main current
git switch main
git fetch upstream
git rebase upstream/main     # or: git merge --ff-only upstream/main
git push origin main
```

> **Tip:** Base each contribution branch on `upstream/main`, not on your fork's possibly stale `main`. It avoids dragging old commits into your pull request.

3.7 Lab: Branch, Conflict, Collaborate

Goal: experience fast-forward and three-way merges, resolve a conflict with zdiff3, and work against a remote using a local bare repository as the "server".

1. Create a bare "server" and two clones simulating two engineers:

`bash cd /tmp && rm -rf lab3 && mkdir lab3 && cd lab3 git init --bare server.git git clone server.git alice && git clone server.git bob`

2. Alice creates the initial content:

`bash cd /tmp/lab3/alice printf "replicas: 2\nimage: app:1.0\n" > values.yaml git add values.yaml && git commit -m "feat: initial values" git push -u origin main`

3. Bob pulls, then both edit the same line:

```bash cd /tmp/lab3/bob && git pull git switch -c feature/scale sed -i.bak 's/replicas: 2/replicas: 4/' values.yaml && rm values.yaml.bak git commit -am "feat: scale to 4 replicas" git push -u origin feature/scale

cd /tmp/lab3/alice sed -i.bak 's/replicas: 2/replicas: 3/' values.yaml && rm values.yaml.bak git commit -am "feat: scale to 3 replicas" git push ```

4. Alice merges Bob's branch and hits a conflict:

`bash git fetch git log --oneline --graph --all git -c merge.conflictStyle=zdiff3 merge origin/feature/scale cat values.yaml`

Resolve to `replicas: 4`, then:

`bash git add values.yaml git merge --continue git log --oneline --graph git push`

5. Observe a fast-forward: Bob updates his `main`.

`bash cd /tmp/lab3/bob && git switch main && git pull --ff-only`

6. Practise a rejected push and a safe force push:
```

```
bash # Bob commits to main without pulling Alice's next change cd /tmp/lab3/alice && echo "# owner: alice" >> values.yaml && git commit -am "docs: owner" && git push cd /tmp/lab3/bob && echo "# team: sre" >> values.yaml && git commit -am "docs: team" git push # rejected git pull --rebase && git push
```

7. Clean up merged branches: `git branch --merged main` and `git push origin --delete feature/scale`.

3.8 Key Takeaways

- A branch is a 41-byte pointer to a commit; HEAD points to the current branch. That is why branching is cheap.
- Fast-forward merges just move a pointer; three-way merges compare base, ours and theirs and create a merge commit.
- `--no-ff`, `--ff-only` and `--squash` let you choose the shape of history deliberately.
- Enable `zdiff3` so conflict markers include the base version; resolve, `git add`, then `--continue`.
- `origin/main` is a local snapshot of the remote, updated only by fetch. `git fetch` is always safe.
- A normal push only fast-forwards; rewritten branches need `--force-with-lease` (plus `--force-if-includes`), never bare `--force` on shared branches.
- In the fork workflow, `origin` is your fork and `upstream` is the canonical repository; base contribution branches on `upstream/main`.

Part 4 — Rewriting History, Undoing and Recovery

Git lets you rewrite local history freely and makes it hard to lose committed work permanently. This Part covers rebase in depth, cherry-pick, the three undo tools (restore, reset, revert), the reflog safety net, stash, and tags with semantic versioning.

The golden rule to keep in mind throughout:

> **Warning:** Do not rewrite history that other people have based work on. Rebasing, amending or resetting a branch that is already pushed and shared forces everyone else to repair their clones. Rewriting your own unshared feature branch is fine and encouraged.

4.1 Rebase

`git rebase <base>` takes the commits on your current branch that are not in `<base>`, and replays them one by one on top of `<base>`.

```
Before (on feature):                After git rebase main:

  A---B---E---F  main                 A---B---E---F  main
       \                                           \
        C---D  feature                              C'---D'  feature
```

`C'` and `D'` contain the same changes as `C` and `D` but are **new commits** with new hashes, because their parent changed. The originals still exist (reachable through the reflog) until garbage collection.

```
git switch feature
git rebase main
```

```
Successfully rebased and updated refs/heads/feature.
```

If a replayed commit conflicts, Git stops:

```
# fix files
git add <files>
git rebase --continue
# or skip this commit entirely
git rebase --skip
# or give up and restore the original branch
git rebase --abort
```

4.1.1 Why rebase?

- **Linear history**: no merge bubbles from "sync with main" merges. `git log` reads like a story.
- **Clean pull requests**: your branch sits on top of current main, so reviewers see only your changes and CI tests what will actually land.
- **Easier bisect and revert**: one linear sequence of meaningful commits.

4.1.2 Interactive rebase

Interactive rebase is your editing suite for local history: reorder, squash, reword, split or drop commits before you share them.

```
git rebase -i HEAD~4        # edit the last four commits
git rebase -i main          # edit everything since branching from main
```

The editor opens with a todo list (oldest first):

```
pick 3a1f2c0 feat(helm): add PDB template
pick 7b9e114 wip
pick c2d0a88 fix typo
pick e41f6d3 feat(helm): expose PDB minAvailable in values

# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
```

Edit it to:

```
pick 3a1f2c0 feat(helm): add PDB template
fixup 7b9e114 wip
fixup c2d0a88 fix typo
pick e41f6d3 feat(helm): expose PDB minAvailable in values
```

Save and close. The result is two meaningful commits instead of four.

Action	Use
`reword`	Fix a commit message
`squash`	Combine with previous, edit combined message
`fixup`	Combine with previous, discard this message

Action	Use
`edit`	Stop to amend or split the commit
`drop`	Delete the commit
`exec`	Run a command after a commit, e.g. tests
reorder lines	Reorder commits

Splitting a commit: mark it `edit`, then when the rebase stops:

```
git reset HEAD~1            # undo the commit, keep changes unstaged
git add -p                  # stage first logical part
git commit -m "refactor: extract helper"
git add -A
git commit -m "feat: use helper in deploy script"
git rebase --continue
```

Test every commit so bisect works later:

```
git rebase -i --exec "make test" main
```

4.1.3 Autosquash

Instead of remembering which commit a fix belongs to, record it at commit time:

```
git commit --fixup=3a1f2c0          # creates "fixup! feat(helm): add PDB template"
git commit --squash=3a1f2c0         # same, but lets you edit the message
git commit --fixup=reword:3a1f2c0   # only change the message of 3a1f2c0
git rebase -i --autosquash main
```

With `--autosquash`, Git moves each `fixup!` commit directly under its target and marks it `fixup` in the todo list. Make it the default:

```
git config --global rebase.autoSquash true
```

This is the cleanest way to address code review comments: push fixup commits so reviewers see exactly what changed, then autosquash before merging.

4.1.4 rebase --onto

`--onto` transplants a range of commits onto a new base. The form is:

```
git rebase --onto <newbase> <upstream> <branch>
```

“Take the commits in `<branch>` that are not in `<upstream>`, and replay them onto `<newbase>`.”

Classic case: you branched `feature-b` off `feature-a`, but `feature-b` should be based on `main`:

```
Before:                                After git rebase --onto main feature-a feature-b:

  A---B  main                            A---B  main
       \                                      \
        C---D  feature-a                       C---D  feature-a
             \                                  \
              E---F  feature-b                   (E'---F' now on B)

                                         A---B---E'---F'  feature-b
```

Another case: drop commits from the middle of a branch.

```
# remove commits C and D (HEAD~4..HEAD~2) from the current branch
git rebase --onto HEAD~4 HEAD~2
```

4.1.5 Stacked branches: --update-refs

When you maintain a stack of dependent branches (`part-1` <- `part-2` <- `part-3`), rebasing the top one used to leave the lower branches pointing at the old commits. Since Git 2.38:

```
git switch part-3
git rebase --update-refs main
```

Git moves every branch that points into the rebased range. Set `rebase.updateRefs=true` to make it the default.

4.2 Merge vs Rebase: The Trade-offs

Aspect	Merge	Rebase
History shape	True, non-linear; shows when work was integrated	Linear; looks as if work happened sequentially
Commit hashes	Preserved	Rewritten for replayed commits
Safe on shared branches	Yes	No (requires force push)
Conflict resolution	Once, in the merge commit	Possibly once per replayed commit (mitigated by rerere)
Traceability of “which PR brought this”	Merge commit groups it	Lost unless you use merge commits at the end
Bisect and log readability	Harder with many merge bubbles	Easier
Reverting a whole feature	`git revert -m 1 <merge>`	Revert each commit, or squash on merge

A pragmatic team policy that gets the best of both:

1. **Rebase** your own feature branch on main while developing (`git pull --rebase`, `git rebase main`).
2. **Clean up** with interactive rebase before review or before merge.
3. **Integrate** into main through the platform using either a merge commit (`--no-ff`) or squash merge, never by rewriting main.

> **Interview:** "When would you not rebase?" When the branch is shared and others have built on it; when you need an exact record of when integration happened (some regulated environments); or when a long-lived branch would require resolving the same conflict across dozens of replayed commits, where a single merge is simpler.

4.3 Cherry-pick

`git cherry-pick` applies the change introduced by an existing commit as a new commit on the current branch. It is how you backport a fix to a release branch without merging everything else from main.

```
A---B---C---F---G  main          (F is a security fix)
     \
      D---E  release/2.4

git switch release/2.4
git cherry-pick F

      D---E---F'  release/2.4
```

```
git switch release/2.4
git cherry-pick -x 4e7d0c2          # -x appends "(cherry picked from commit ...)"
git cherry-pick A..B                # range: commits after A through B
git cherry-pick --no-commit 4e7d0c2 # apply changes, let me edit before committing
git cherry-pick -m 1 <merge-sha>    # cherry-pick a merge, relative to its first parent
git cherry-pick --continue | --skip | --abort
```

> **Tip:** Always use `-x` when backporting to release branches. The trailer records where the fix came from, so auditors and future engineers can trace it, and tooling can detect which fixes are already backported.

Downsides: the cherry-picked commit is a duplicate with a different hash. If the branches are later merged, Git usually handles identical changes cleanly, but heavy cherry-picking between long-lived branches is a sign the branching strategy needs attention (Part 6).

4.4 Undoing Things: restore, reset, revert

These three are often confused. The key questions are “which tree do I want to change?” and “has this been shared?”.

Command	Changes history?	Safe on shared branches?	Typical use
`git restore`	No	Yes	Discard or unstage file changes
`git reset`	Yes, moves the branch pointer	No	Undo local commits, reshape local history
`git revert`	No, adds a new commit	Yes	Undo a pushed commit publicly

4.4.1 git reset: soft, mixed, hard

`git reset <commit>` moves the current branch to `<commit>`. The mode decides what happens to the index and working directory.

Mode	Branch (HEAD)	Index	Working dir	Result
`--soft`	Moved	Unchanged	Unchanged	Undone commits’ changes are staged
`--mixed` (default)	Moved	Reset to commit	Unchanged	Changes are unstaged but on disk
`--hard`	Moved	Reset to commit	Reset to commit	Changes are gone (from the working tree)
`--keep`	Moved	Reset	Reset, but aborts if local edits would be lost	Safer alternative to hard

```
Start:   A---B---C---D  main (HEAD)

git reset --soft B    ->  A---B  main;  index and files still contain C+D changes (staged)
git reset --mixed B   ->  A---B  main;  files contain C+D changes (unstaged)
git reset --hard B    ->  A---B  main;  files match B exactly
```

Common recipes:

```
git reset --soft HEAD~3 && git commit -m "feat: one clean commit"   # squash last 3
git reset HEAD~1                    # undo last commit, keep changes to re-stage
git reset --hard origin/main        # make my branch exactly match the remote
git reset -p HEAD file              # unstage selected hunks
```

> **Warning:** `git reset --hard` discards uncommitted changes permanently. Committed work it "removes" is still recoverable through the reflog, but edits that were never committed or stashed are not. Run `git status` and `git stash` first if in doubt.

4.4.2 git revert

`git revert` creates a new commit that applies the inverse of an existing commit. History is preserved, so it is the correct way to undo something already on a shared branch, such as a bad deploy to `main`.

```
git revert 9f2c1ab
git revert --no-edit HEAD
git revert HEAD~3..HEAD             # revert a range (creates one revert per commit)
git revert --no-commit HEAD~3..HEAD && git commit -m "revert: roll back release 2.4 changes"
```

Reverting a merge commit requires telling Git which parent is the mainline:

```
git revert -m 1 4e7d0c2             # undo everything the merged branch brought in
```

> **Warning:** After reverting a merge, re-merging the same branch later will not bring the changes back, because Git sees those commits as already merged. To re-land, revert the revert (`git revert <revert-sha>`) and then merge any new fixes.

4.4.3 Choosing the right tool

```
Is the change committed?
 |-- No  -> git restore (discard) / git restore --staged (unstage) / git stash (park)
 |-- Yes -> Has it been pushed to a shared branch?
            |-- No  -> git reset, git commit --amend, git rebase -i
            |-- Yes -> git revert
```

4.5 The Reflog: Your Safety Net

Every time a branch tip or HEAD moves (commit, reset, rebase, switch, merge, pull), Git records the old and new positions in the **reflog**. It is local to your clone and by default keeps entries for 90 days (30 days for unreachable commits).

```
git reflog
```

```
e41f6d3 (HEAD -> feature) HEAD@{0}: rebase (finish): returning to refs/heads/feature
e41f6d3 HEAD@{1}: rebase (pick): feat(helm): expose PDB minAvailable
3a1f2c0 HEAD@{2}: rebase (start): checkout main
b77c9d2 HEAD@{3}: commit: feat(helm): expose PDB minAvailable
```

```
5d12aa0 HEAD@{4}: commit: wip
3c1d2e9 HEAD@{5}: checkout: moving from main to feature
```

```
git reflog show feature            # reflog for one branch
git log -g --oneline               # reflog as a log
git show HEAD@{3}
git diff HEAD@{1} HEAD
```

4.5.1 Recovery recipes

Undo a bad rebase or reset:

```
git reflog                          # find the entry before the operation, e.g. HEAD@{3}
git reset --hard HEAD@{3}
```

Git also writes `ORIG_HEAD` before risky operations (rebase, reset, merge), so `git reset --hard ORIG_HEAD` often undoes the last one directly.

Recover a deleted branch:

```
git branch -D old-spike
```

```
Deleted branch old-spike (was a77e010).
```

```
git branch old-spike a77e010        # hash from the message or from git reflog
```

Find commits not referenced by any reflog (e.g. a dropped stash):

```
git fsck --lost-found --no-reflogs
git fsck --unreachable | grep commit
```

> **Note:** The reflog is per clone and is not pushed. If you rewrote history on a laptop that has since been wiped, the server and teammates' clones (and their reflogs) are your fallback. Hosting platforms also keep push event logs and may restore force-pushed branches via support or API.

4.6 Stash

`git stash` saves uncommitted changes (staged and unstaged) on a stack and cleans your working directory, so you can switch context quickly.

```
git stash push -m "wip: ingress tuning"   # save tracked changes
git stash -u                              # include untracked files
```

```
git stash -a                              # include ignored files too
git stash push -p                         # stash selected hunks
git stash push -- values.yaml             # stash a single path
git stash list
git stash show -p stash@{1}
git stash pop                             # apply latest and drop it
git stash apply stash@{1}                 # apply, keep in list
git stash drop stash@{1}
git stash branch fix/ingress stash@{0}    # new branch from where the stash was made, then
apply
git stash clear                           # delete all (careful)
```

```
stash@{0}: On feature/pdb: wip: ingress tuning
stash@{1}: WIP on main: 9f2c1ab ci: cache Go modules
```

Use stash for short interruptions ("prod is on fire, I need main now"). For anything longer than a day, commit to a WIP branch instead; stashes are easy to forget and are not pushed.

> **Tip:** `git stash branch` is the cleanest fix when `stash pop` conflicts because the code moved on. It recreates the original context.

4.7 Tags, Semantic Versioning and Releases

4.7.1 Lightweight vs annotated tags

A tag is a fixed name for a specific commit. Unlike branches, tags do not move.

Type	Stores	Use
Lightweight	Just a ref pointing at a commit	Private or temporary markers
Annotated	A tag object with tagger, date, message, optional signature	Releases (always)

```
git tag v1.4.0                                # lightweight
git tag -a v1.4.0 -m "Release 1.4.0"          # annotated
git tag -s v1.4.0 -m "Release 1.4.0"          # signed annotated
git tag -a v1.3.1 4e7d0c2 -m "Hotfix release" # tag an older commit
git tag -l "v1.*"                             # list
git show v1.4.0
git push origin v1.4.0                        # push one tag
git push --follow-tags                        # push annotated tags reachable from pushed
commits
git tag -d v1.4.0 && git push origin --delete v1.4.0   # delete locally and remotely
git describe --tags                           # e.g. v1.4.0-3-g9f2c1ab
```

`git describe` output `v1.4.0-3-g9f2c1ab` means "three commits after v1.4.0, at commit 9f2c1ab", which is ideal for embedding build versions:

```
VERSION=$(git describe --tags --always --dirty)
docker build -t registry.acme.io/api:"$VERSION" .
```

> **Warning:** Never move or reuse a published release tag. Consumers, caches, Go module proxies and artifact registries assume tags are immutable. If v1.4.0 is broken, release v1.4.1.

4.7.2 Semantic Versioning

SemVer uses `MAJOR.MINOR.PATCH`:

Part	Increment when	Example
MAJOR	Incompatible API change	1.9.3 -> 2.0.0
MINOR	Backward-compatible new functionality	1.9.3 -> 1.10.0
PATCH	Backward-compatible bug fix	1.9.3 -> 1.9.4
Pre-release	Unstable preview	2.0.0-rc.1, 2.0.0-beta.2
Build metadata	Ignored for precedence	2.0.0+build.417

Versions below 1.0.0 signal that anything may change. Combined with Conventional Commits (Part 2), the next version can be computed from commit messages: any `!` or `BREAKING CHANGE` means major, any `feat` means minor, otherwise `fix` / `perf` means patch.

4.7.3 Automating releases

Tools such as semantic-release, release-please and git-cliff read commit history since the last tag, compute the next version, generate a changelog, create the tag and publish a platform release. A typical pipeline:

```
merge to main
   -> CI parses commits since last tag (feat, fix, BREAKING CHANGE)
   -> computes v1.5.0, updates CHANGELOG.md
   -> creates annotated tag v1.5.0 and GitHub/GitLab release
   -> tag push triggers the build-and-publish pipeline (Part 7)
```

4.8 Lab: Rewrite, Break and Recover

Goal: clean up a messy branch with interactive rebase and autosquash, backport with cherry-pick, then deliberately destroy work and recover it.

1. Set up:

```bash
cd /tmp && rm -rf lab4 && mkdir lab4 && cd lab4 && git init
echo "base" > app.txt && git add . && git commit -m "chore: init"
git switch -c feature/pdb
echo "pdb template" > pdb.yaml && git add . && git commit -m "feat(helm): add PDB template"
echo "values" > values.yaml && git add . && git commit -m "feat(helm): expose PDB values"
```

2. Make a fixup for the first feature commit and autosquash:

```bash
echo "pdb template v2" > pdb.yaml
git commit -a --fixup=HEAD~1 # targets "add PDB template"
git log --oneline
git rebase -i --autosquash main # inspect the todo list, then save it unchanged
git log --oneline # two clean commits
```

3. Backport: create a release branch and cherry-pick only the template commit:

```bash
git switch -c release/1.0 main
git cherry-pick -x feature/pdb~1
git log -1
```

4. Tag the release and describe it:

```bash
git tag -a v1.0.0 -m "Release 1.0.0"
echo "fix" >> app.txt && git commit -am "fix: patch"
git describe --tags
```

5. Destroy and recover:

```bash
git switch feature/pdb
git reset --hard main # "lose" both feature commits
git log --oneline # gone
git reflog -n 5
git reset --hard HEAD@{1} # back
git log --oneline
```

6. Delete a branch and resurrect it:

```bash
git switch main
git branch -D feature/pdb # note the hash in the output
git branch feature/pdb <hash>
```

7. Revert publicly: on main, make a commit, then `git revert --no-edit HEAD` and inspect `git log -p -2`.

8. Practise stash: edit `app.txt`, run `git stash -u -m "wip"`, switch branches, come back, `git stash pop`.

4.9 Key Takeaways

- Rebase replays commits onto a new base, producing new hashes; use it on your own branches, never on shared ones.
- Interactive rebase, `--fixup` plus `--autosquash`, `--onto` and `--update-refs` give you precise control over local history.
- Rebase while developing, merge (or squash merge) into main through the platform.
- `cherry-pick -x` is the standard backport tool; heavy cherry-picking signals a branching problem.
- `restore` changes files, `reset` moves branches (soft/mixed/hard), `revert` undoes publicly with a new commit.
- The reflog and `ORIG_HEAD` make almost any committed work recoverable for weeks; uncommitted work has no such safety net.
- Stash is for short interruptions; commit WIP branches for anything longer.

- Release with annotated, immutable tags following SemVer, and automate version bumps from Conventional Commits.

Part 5 — Git Internals and Power Tools

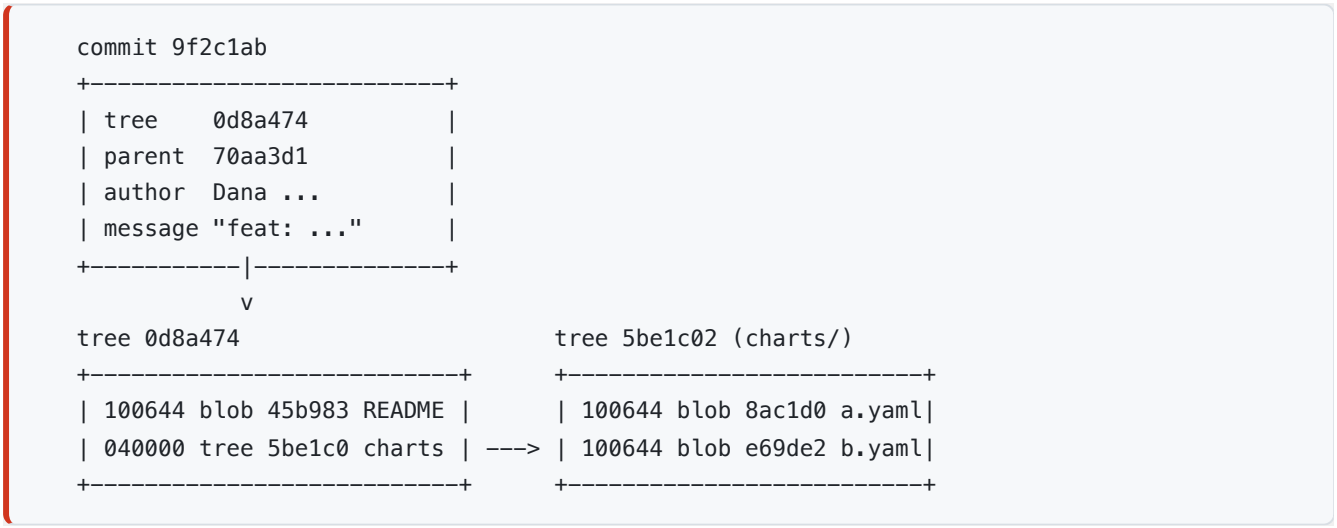
Advanced Git troubleshooting is much easier once you understand what Git actually stores. This Part opens up the object database, then covers the power tools that rely on it: bisect, blame, worktrees, submodules and subtrees, sparse checkout and partial clone for monorepos, Git LFS, and performance tuning for large repositories.

5.1 The Object Database

Git is, at its core, a **content-addressable key-value store**. You give it content; it returns a key (the hash); the content can later be retrieved by that key. Everything else (branches, commits, history) is built on top.

There are four object types:

Type	Represents	Contains
`blob`	File contents	Raw bytes only. No filename, no permissions
`tree`	A directory	List of entries: mode, type, hash, name
`commit`	A snapshot plus metadata	Tree hash, parent hash(es), author, committer, message
`tag`	An annotated tag	Target object hash, type, tag name, tagger, message, optional signature



5.1.1 Hashing content yourself

```
echo "hello" | git hash-object --stdin
```

```
ce013625030ba8dba906f756967f9e9ca394464a
```

The hash is SHA-1 over a header (`blob 6\0`) plus the content. The same content produces the same hash in every repository in the world, which is why identical files are stored once and why Git can tell instantly whether two trees are equal.

5.1.2 Exploring with git cat-file

`git cat-file` is the X-ray tool. `-t` prints the type, `-p` pretty-prints the content, `-s` the size.

```
git cat-file -p HEAD
```

```
tree 0d8a474fc67971fb3dd7616e26323d3066442555
parent 70aa3d1f0c5e2b4a9d8e7f6a5b4c3d2e1f0a9b8c
author Dana Ops <dana@example.com> 1728720000 +0000
committer Dana Ops <dana@example.com> 1728720000 +0000

feat(helm): add PDB to ingress chart
```

```
git cat-file -p HEAD^{tree}
```

```
100644 blob 45b983be36b73c0788dc9cbcb76cbb80fc7bb057    README.md
040000 tree 5be1c02a7d3e9f8b6c4a2d1e0f9a8b7c6d5e4f3a    charts
```

```
git cat-file -p 45b983be
git cat-file -t v1.0.0
git cat-file -p v1.0.0
```

```
tag
object 53c9f9cf26fed7ba52c02ac15b9cd4d3037e566a
type commit
tag v1.0.0
tagger Dana Ops <dana@example.com> 1728720000 +0000

Release 1.0.0
```

Note that the annotated tag is its own object pointing at a commit, whereas a lightweight tag is just a ref.

5.1.3 Why this matters

- **A commit hash covers everything.** Commit -> tree -> blobs, plus parent hashes. That is a Merkle tree, so one commit hash cryptographically pins the whole history. It is also why rebasing or amending changes hashes.
- **Renames are not stored.** A tree entry is just a name pointing at a blob; Git infers renames by comparing blobs.
- **Empty directories cannot be committed.** A tree only exists if it contains entries, which is why people commit `.gitkeep` files.

- **File mode matters.** `100755` vs `100644` is part of the tree; `chmod +x` on a script is a real change.

Mode	Meaning
`100644`	Regular file
`100755`	Executable file
`120000`	Symbolic link
`040000`	Tree (directory)
`160000`	Gitlink (submodule commit)

5.1.4 Plumbing vs porcelain

Commands like `commit`, `log`, `switch` are **porcelain**: friendly, output may change between versions. Commands like `cat-file`, `hash-object`, `rev-parse`, `update-ref`, `for-each-ref`, `rev-list` are **plumbing**: stable, designed for scripts. In CI scripts, prefer plumbing or porcelain with stable flags (`--porcelain`, `--format`).

```
git rev-parse HEAD                    # full hash of HEAD
git rev-parse --abbrev-ref HEAD       # current branch name
git rev-parse --show-toplevel         # repo root
git status --porcelain=v2             # stable machine-readable status
git for-each-ref --format='%(refname:short) %(committerdate:relative)' refs/heads/
git rev-list --count main             # number of commits
```

> **Interview:** "SHA-1 is broken; is Git insecure?" Git uses a hardened SHA-1 implementation that detects known collision attacks, and Git supports SHA-256 repositories (`git init --object-format=sha256`), though hosting support is still limited. Commit and tag signing (Part 6) adds identity on top of integrity.

5.2 Refs, Packfiles and Garbage Collection

5.2.1 Refs

A ref is a name for a hash.

Ref	Location
Branches	`refs/heads/<name>`
Remote-tracking	`refs/remotes/<remote>/<name>`
Tags	`refs/tags/<name>`

Ref	Location
Stash	`refs/stash`
Notes	`refs/notes/commits`
PR heads (GitHub)	`refs/pull/<n>/head` on the server
MR heads (GitLab)	`refs/merge-requests/<n>/head` on the server
Symbolic	`HEAD` -> `ref: refs/heads/main`

Loose refs are individual files; `git pack-refs` consolidates them into `.git/packed-refs`. Newer Git (2.45+) also offers the **reftable** backend (`git init --ref-format=reftable`), which scales far better for repositories with hundreds of thousands of refs.

```
git update-ref refs/heads/rescue 3c1d2e9    # plumbing: create a branch
git symbolic-ref HEAD                        # refs/heads/main
```

Fetching a pull request locally uses those server-side refs:

```
git fetch origin pull/123/head:pr-123
git switch pr-123
```

5.2.2 Loose objects and packfiles

New objects are first written as individual zlib-compressed **loose objects** in `.git/objects/ab/cdef...`. Periodically, Git packs them into a **packfile** (`.pack`) with an index (`.idx`). Inside a pack, similar objects are stored as deltas against each other, which is how a repo with thousands of versions of a file stays small.

```
git count-objects -vH
```

```
count: 0
size: 0 bytes
in-pack: 48213
packs: 1
size-pack: 61.20 MiB
prune-packable: 0
garbage: 0
size-garbage: 0 bytes
```

```
git verify-pack -v .git/objects/pack/pack-*.idx | sort -k3 -n | tail -5   # largest objects
```

5.2.3 Garbage collection and maintenance

Objects not reachable from any ref or reflog are eventually pruned by `git gc`. Defaults: reflog entries expire after 90 days, unreachable ones after 30 days, and unreachable loose objects are pruned after 2 weeks.

```
git gc                      # pack, prune, pack refs
git gc --aggressive         # slower, tighter deltas; rarely needed
git prune --dry-run
git maintenance start       # schedule background maintenance (prefetch, commit-graph, loose-objects, incremental-repack)
```

`git maintenance start` registers the repository with the OS scheduler (launchd, systemd timers, cron or Windows Task Scheduler) and runs incremental tasks hourly and daily, keeping large repositories fast without long `gc` pauses.

5.3 git bisect: Binary Search for Bugs

When you know a version that works and a version that is broken, `git bisect` finds the first bad commit in O(log n) steps. 1,000 commits take about 10 tests.

```
git bisect start
git bisect bad                  # current commit is broken
git bisect good v1.3.0          # this tag was fine
```

```
Bisecting: 37 revisions left to test after this (roughly 5 steps)
[a1b2c3d] refactor(api): move retry config
```

Test, then mark:

```
git bisect good       # or: git bisect bad, or: git bisect skip (cannot test this one)
```

Eventually:

```
c2d0a88e1f is the first bad commit
commit c2d0a88e1f
Author: Lee Chen <lee@example.com>
    perf(api): reduce connection pool size
```

```
git bisect log > bisect.log     # save the session
git bisect reset                # return to where you started
```

5.3.1 Automated bisect

If a script can detect the bug, let Git run it. Exit code 0 means good, 1 to 127 (except 125) means bad, 125 means skip.

```
git bisect start HEAD v1.3.0
git bisect run ./scripts/check-healthz.sh
```

```
#!/usr/bin/env bash
# scripts/check-healthz.sh
make build >/dev/null 2>&1 || exit 125     # cannot build: skip
./bin/api --selftest | grep -q "pool ok"
```

You can rename the terms for non-bug searches, for example finding when performance changed:

```
git bisect start --term-old=fast --term-new=slow
```

> **Tip:** Bisect works best on a history where every commit builds and passes tests. That is a concrete, practical reason to keep commits atomic and to use `git rebase -i --exec "make test"` before merging.

5.4 git blame: Who Changed This Line and Why

```
git blame -L 40,60 deploy/values.yaml
```

```
3a1f2c0e (Dana Ops 2025-03-02 10:14:22 +0000 40) resources:
c2d0a88e (Lee Chen 2025-05-19 16:02:51 +0000 41)   limits:
c2d0a88e (Lee Chen 2025-05-19 16:02:51 +0000 42)     memory: 256Mi
```

Option	Purpose
`-L 40,60` / `-L :funcname`	Limit to lines or a function
`-w`	Ignore whitespace changes
`-M` / `-C` / `-C -C -C`	Detect lines moved within a file / copied from other files
`--ignore-rev <sha>`	Skip a commit (e.g. mass reformatting)
`--ignore-revs-file .git-blame-ignore-revs`	Skip a list of commits
`-e`	Show emails

After a large automatic reformat (prettier, black, terraform fmt), add its hash to `.git-blame-ignore-revs` and configure:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

GitHub and GitLab honour this file in their web blame views too.

Blame tells you which commit last touched a line, not who is “at fault”. The real value is the commit message and the linked pull request: `git show <sha>` usually answers “why is this memory limit 256Mi?”.

5.5 Worktrees: Multiple Branches Checked Out at Once

A worktree is an additional working directory attached to the same repository. Each worktree has its own HEAD, index and files, but they share the object database and refs.

```
git worktree add ../infra-hotfix -b hotfix/cve origin/main
git worktree add ../infra-review origin/feature/pdb --detach
git worktree list
```

```
/home/dana/infra          9f2c1ab [main]
/home/dana/infra-hotfix   9f2c1ab [hotfix/cve]
/home/dana/infra-review   3c1d2e9 (detached HEAD)
```

```
git worktree remove ../infra-review
git worktree prune             # clean up records of manually deleted worktrees
```

Why use worktrees instead of stash or a second clone?

- Fix a production issue without disturbing a half-finished feature or a running local dev server.
- Review a colleague’s PR side by side with your own work.
- Run a long test suite on one branch while coding on another.
- Cheaper than a second clone: no duplicated objects, no second fetch.

> **Note:** A branch can be checked out in only one worktree at a time. Git refuses a second checkout to prevent two working directories from fighting over one branch.

5.6 Submodules vs Subtrees

Both embed one repository inside another. They make opposite trade-offs.

5.6.1 Submodules

A submodule records a **pointer** (a gitlink, mode `160000`) to a specific commit of another repository, plus its URL in `.gitmodules`.

```
git submodule add git@github.com:acme/terraform-modules.git modules/shared
git commit -m "build: add shared terraform modules as submodule"
```

```
# .gitmodules
[submodule "modules/shared"]
    path = modules/shared
    url = git@github.com:acme/terraform-modules.git
    branch = main
```

```
git clone --recurse-submodules git@github.com:acme/platform-infra.git
git submodule update --init --recursive     # after a normal clone
git submodule update --remote modules/shared # move pointer to latest of tracked branch
git add modules/shared && git commit -m "build: bump shared modules"
git config --global submodule.recurse true   # make switch/pull update submodules
```

Pain points: forgetting `--recurse-submodules` leaves empty directories; a submodule checkout is in detached HEAD by default, so commits made there are easy to lose; CI needs credentials for every submodule repository.

5.6.2 Subtrees

`git subtree` copies another repository's content (optionally with its history) into a subdirectory of yours. Consumers see normal files; no special clone steps.

```
git subtree add --prefix=modules/shared git@github.com:acme/terraform-modules.git main --
squash
git subtree pull --prefix=modules/shared git@github.com:acme/terraform-modules.git main --
squash
git subtree push --prefix=modules/shared git@github.com:acme/terraform-modules.git
feature/fix
```

5.6.3 Comparison

Aspect	Submodule	Subtree
What is stored	Pointer to external commit	Actual files (and optionally history)
Clone experience	Needs `--recurse-submodules`	Just works
Pinning exact version	Explicit, precise	Implicit in merged content
Contributing back upstream	Natural (it is a real repo)	Possible with `subtree push`, more awkward
Repo size	Small	Grows with embedded content
CI complexity	Extra auth and checkout steps	None

Aspect	Submodule	Subtree
Best for	Large, independently versioned dependencies	Vendoring small shared code

> **Tip:** For many DevOps use cases neither is ideal. Terraform modules, Helm charts and Ansible roles are often better consumed through their own registries or versioned references (`source = "git::https://...?ref=v1.4.0"`) than embedded in Git.

5.7 Monorepos: Sparse Checkout and Partial Clone

Monorepos hold many services in one repository. They simplify atomic cross-service changes and shared tooling but stress Git with large histories and file counts. Git has several features designed for this.

5.7.1 Shallow clone

```
git clone --depth 1 https://github.com/acme/monorepo.git
git fetch --deepen 50          # get more history later
git fetch --unshallow          # convert to a full clone
```

Shallow clones truncate **history**. They are ideal for one-off CI builds but break commands that need history (`describe` , `log` across ranges, merge-base calculations).

5.7.2 Partial clone

Partial clone keeps full history but omits objects until needed.

Filter	Downloads up front	Fetches on demand
`--filter=blob:none`	All commits and trees	File contents (blobs) when checked out
`--filter=tree:0`	Commits only	Trees and blobs
`--filter=blob:limit=1m`	Everything except blobs over 1 MB	Large blobs

```
git clone --filter=blob:none git@github.com:acme/monorepo.git
```

`blob:none` (a “blobless clone”) is usually the best default for developers on large repos: `git log` and `git merge-base` work fully offline, and only the files you actually check out are downloaded. `tree:0` (“treeless”) suits throwaway CI builds that need history but few files.

5.7.3 Sparse checkout

Sparse checkout limits which files appear in your working directory. Cone mode (the default) works on directories and is fast.

```
git clone --filter=blob:none --sparse git@github.com:acme/monorepo.git
cd monorepo
git sparse-checkout set services/payments libs/common deploy/payments
git sparse-checkout add services/auth
git sparse-checkout list
git sparse-checkout disable
```

Combined, partial clone plus sparse checkout means you download only the history metadata plus the files of the directories you work on. On a multi-gigabyte monorepo this can turn a 20-minute clone into under a minute.

5.7.4 Scalar

`scalar` ships with Git and configures a repository with the recommended large-repo settings (partial clone, sparse checkout, commit-graph, multi-pack index, background maintenance, file system monitor):

```
scalar clone https://github.com/acme/monorepo.git
scalar register           # apply to an existing clone
```

5.8 Large Files: Git LFS

Git stores every version of every file forever, and each clone downloads all of it. Committing a 500 MB model or ISO a few times makes the repository unusable. Git LFS (Large File Storage) replaces large files in Git with small pointer files and stores the real content on an LFS server.

```
git lfs install                       # once per machine: sets up filters and hooks
git lfs track "*.tar.gz" "*.onnx" "assets/**"
git add .gitattributes
git add model.onnx
git commit -m "feat: add inference model"
git push
```

`.gitattributes` gets:

```
*.tar.gz filter=lfs diff=lfs merge=lfs -text
*.onnx filter=lfs diff=lfs merge=lfs -text
```

What Git actually stores is a pointer:

```
version https://git-lfs.github.com/spec/v1
oid sha256:4d7a214614ab2935c943f9e0ff69d22eadbb8f32b1258daaa5e2ca24d17e2393
size 524288000
```

Command	Purpose
`git lfs ls-files`	List LFS-managed files
`git lfs pull`	Download LFS content for current checkout
`git lfs fetch --all`	Download all LFS objects (for mirroring)
`git lfs migrate import --include="*.zip" --everything`	Rewrite history to move existing files into LFS
`git lfs prune`	Delete old local LFS copies
`GIT_LFS_SKIP_SMUDGE=1 git clone ...`	Clone without downloading LFS content

> **Warning:** LFS is not a substitute for an artifact repository. Build outputs, container images and packages belong in Artifactory, Nexus, a container registry or S3, not in Git. Also, hosting plans meter LFS storage and bandwidth; CI pipelines that pull LFS on every run can burn through quotas.

5.9 Performance for Big Repositories

Symptom	Remedy
`git status` slow on many files	`git config core.fsmonitor true` (built-in FS monitor on macOS/Windows) and `core.untrackedCache true`
`git log --graph` / merge-base slow	`git commit-graph write --reachable` or `fetch.writeCommitGraph true`
Many packfiles	`git maintenance start`, or `git repack -d --write-midx`
Clone is huge	Partial clone, sparse checkout, shallow in CI
Huge objects in history	Find with `git rev-list --objects --all` plus `cat-file --batch-check`; move to LFS or remove with filter-repo
Thousands of refs	`git pack-refs --all`, reftable backend, prune stale branches
Slow fetch in CI	Reuse caches, `--depth`, `--filter`, fetch only needed refs

Finding the biggest blobs in history:

```
git rev-list --objects --all \
  | git cat-file --batch-check='%(objecttype) %(objectname) %(objectsize) %(rest)' \
  | awk '$1 == "blob"' | sort -k3 -n -r | head -10
```

```
blob 8f3e1a... 214958080 vendor/old-installer.iso
blob 1c22b7... 52428800 data/dump.sql
blob 77ad0e... 10485760 assets/video.mp4
```

Useful diagnostics:

```
GIT_TRACE=1 git status                # what Git is executing
GIT_TRACE_PERFORMANCE=1 git status    # timings
GIT_TRACE_PACKET=1 git fetch          # protocol-level debugging
GIT_CURL_VERBOSE=1 git fetch          # HTTPS debugging
GIT_SSH_COMMAND="ssh -vvv" git fetch  # SSH debugging
```

5.10 Lab: Inside the Machine

Goal: inspect objects by hand, find a regression with automated bisect, and use worktrees and sparse checkout.

1. Build a repository with history:

`bash cd /tmp && rm -rf lab5 && mkdir lab5 && cd lab5 && git init for i in $(seq 1 30); do echo "pool_size=$((i < 18 ? 10 : 2))" > config.ini; echo "$i" > build.txt; git add .; git commit -qm "chore: build $i"; done git log --oneline | head -3`

2. Walk the object graph:

`bash git cat-file -t HEAD git cat-file -p HEAD git cat-file -p HEAD^{tree} git cat-file -p HEAD:config.ini echo "pool_size=2" | git hash-object --stdin # compare with the blob hash in the tree`

3. Bisect automatically. A good build has `pool_size=10`:

`bash git bisect start HEAD HEAD~29 git bisect run sh -c 'grep -q "pool_size=10" config.ini' git bisect reset`

Confirm the reported first bad commit is “build 18”.

4. Pack and inspect:

`bash git count-objects -vH git gc git count-objects -vH ls .git/objects/pack`

5. Worktrees:

`bash git worktree add ../lab5-hotfix -b hotfix/pool HEAD~20 cat ../lab5-hotfix/config.ini git worktree list git worktree remove ../lab5-hotfix`

6. Sparse checkout:

`bash mkdir -p services/a services/b && echo a > services/a/x && echo b > services/b/y git add . && git commit -qm "feat: services" git sparse-checkout set services/a ls services # only a git sparse-checkout disable`

7. Optional: install Git LFS, track `*.bin`, commit a 5 MB random file (`head -c 5000000 /dev/urandom > big.bin`) and run `git cat-file -p HEAD:big.bin` to see the pointer.

5.11 Key Takeaways

- Git stores four object types: blobs (content), trees (directories), commits (snapshot + metadata) and annotated tags, each named by the hash of its content.
- A commit hash pins the entire history (a Merkle tree); this is why rewriting changes hashes.
- Refs are names for hashes; packfiles delta-compress objects; `git maintenance` keeps large repos healthy.
- `git bisect run` finds regressions automatically in logarithmic time, if commits are atomic and buildable.
- `git blame` with `-w`, `-C` and `.git-blame-ignore-revs` leads you to the commit and PR that explain a line.
- Worktrees give you several branches checked out at once from one repository.
- Submodules pin external commits precisely but complicate clones; subtrees vendor files simply.
- For monorepos, combine blobless partial clone with cone-mode sparse checkout, or use `scalar`.
- Use Git LFS for large binaries that must be versioned, and artifact repositories for build outputs.

Part 6 — Hooks, Signing and Team Workflows

Individual Git skill is only half the job. A DevOps engineer is usually the person who decides how a whole team uses Git: what gets checked before a commit, how identity is verified, which branching model fits the release process, and how the hosting platform enforces it. This Part covers hooks, commit signing, branching strategies, pull requests and code review, branch protection, CODEOWNERS and platform specifics.

6.1 Git Hooks

Hooks are scripts Git runs at specific points. Client-side hooks live in `.git/hooks/` (or a directory set by `core.hooksPath`) and run on the developer's machine. Server-side hooks run on the Git server when receiving a push.

6.1.1 Client-side hooks

Hook	Runs	Can abort?	Typical use
`pre-commit`	Before the commit message editor opens	Yes	Lint, format, secret scan on staged files
`prepare-commit-msg`	After default message is created	Yes	Insert ticket ID from branch name
`commit-msg`	After message is written	Yes	Enforce Conventional Commits
`post-commit`	After commit is created	No	Notifications
`pre-rebase`	Before rebase starts	Yes	Prevent rebasing protected branches
`post-checkout` / `post-merge`	After switch / merge	No	Reinstall dependencies, regenerate files
`pre-push`	Before objects are sent	Yes	Run tests, block pushes to main

A minimal `commit-msg` hook enforcing Conventional Commits:

```
#!/usr/bin/env bash
# .githooks/commit-msg
pattern='^(feat|fix|docs|refactor|perf|test|build|ci|chore|revert)(\([a-z0-9-]+\))?!?: .
{1,72}$'
first_line=$(head -n1 "$1")
if ! [[ $first_line =~ $pattern ]] && ! [[ $first_line =~ ^(fixup|squash|amend)! ]]; then
  echo "commit-msg: subject must follow Conventional Commits, got:" >&2
  echo "  $first_line" >&2
```

```
  exit 1
fi
```

A `prepare-commit-msg` hook that prefixes the ticket ID from the branch name:

```
#!/usr/bin/env bash
# .githooks/prepare-commit-msg
branch=$(git rev-parse --abbrev-ref HEAD)
ticket=$(echo "$branch" | grep -oE '[A-Z]+-[0-9]+' | head -1)
if [ -n "$ticket" ] && ! grep -q "$ticket" "$1"; then
  printf '\n\nRefs: %s\n' "$ticket" >> "$1"
fi
```

6.1.2 Sharing hooks with the team

Files in `.git/hooks` are not versioned or cloned. Two common solutions:

```
# 1. Commit hooks to the repo and point Git at them
chmod +x .githooks/*
git config core.hooksPath .githooks
```

```
# 2. Use a hook manager (pre-commit, Husky, Lefthook)
```

> **Warning:** Client-side hooks are a convenience, not a control. Anyone can skip them with `git commit --no-verify` or by not installing them. Anything that must be enforced belongs in CI and server-side rules too.

6.1.3 The pre-commit framework

`pre-commit` (pre-commit.com) is a language-agnostic hook manager. It installs pinned versions of each tool into isolated environments, so every developer and CI job runs exactly the same checks.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
        args: [--allow-multiple-documents]
      - id: check-added-large-files
        args: [--maxkb=1024]
      - id: detect-private-key
      - id: check-merge-conflict
  - repo: https://github.com/gitleaks/gitleaks
```

```
    rev: v8.21.2
    hooks:
      - id: gitleaks
  - repo: https://github.com/antonbabenko/pre-commit-terraform
    rev: v1.96.1
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
      - id: terraform_tflint
  - repo: https://github.com/adrienverge/yamllint
    rev: v1.35.1
    hooks:
      - id: yamllint
  - repo: https://github.com/shellcheck-py/shellcheck-py
    rev: v0.10.0.1
    hooks:
      - id: shellcheck
```

```
pip install pre-commit            # or: brew install pre-commit / pipx install pre-commit
pre-commit install                # installs .git/hooks/pre-commit
pre-commit install --hook-type commit-msg
pre-commit run --all-files        # run everything once, e.g. in CI
pre-commit autoupdate             # bump rev: pins
```

```
trim trailing whitespace.................................................Passed
check yaml...............................................................Passed
check for added large files..............................................Passed
detect private key.......................................................Failed
- hook id: detect-private-key
- exit code: 1

Private key found: deploy/keys/id_rsa
Detect hardcoded secrets.................................................Failed
```

Run the same `pre-commit run --all-files` in CI so the checks are mandatory even for people who never installed the hooks.

> **Note:** The `rev:` values above are examples; pin to the current release of each hook repository and let `pre-commit autoupdate` or Dependabot/Renovate keep them fresh.

6.1.4 Server-side hooks

Hook	Runs	Use
`pre-receive`	Once per push, before any ref is updated	Reject pushes (policy, secrets, size limits)
`update`	Once per ref being updated	Per-branch rules
`post-receive`	After refs are updated	Trigger CI, notify chat, mirror

On self-hosted Git servers (GitLab self-managed, Gitea, bare SSH servers) you can install these directly. On SaaS platforms you configure equivalent behaviour through branch protection, push rules, secret scanning push protection and webhooks.

```
#!/usr/bin/env bash
# pre-receive: reject any pushed blob larger than 10 MB
zero=0000000000000000000000000000000000000000
while read -r old new ref; do
  [ "$new" = "$zero" ] && continue
  range=$([ "$old" = "$zero" ] && echo "$new" || echo "$old..$new")
  git rev-list --objects "$range" \
    | git cat-file --batch-check='%(objecttype) %(objectsize) %(rest)' \
    | awk '$1=="blob" && $2 > 10485760 { print "Rejected large file: " $3; bad=1 } END { exit
bad }' \
    || exit 1
done
```

6.2 Signing Commits and Tags

Anyone can set `user.email` to your address and create commits “authored” by you. Signing proves a commit was created by someone holding a specific private key. Platforms then display a “Verified” badge, and branch protection can require signatures.

6.2.1 SSH signing (simplest, Git 2.34+)

You probably already have an SSH key:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

Upload the same public key to GitHub as a **Signing Key** (separate from the authentication key entry, even if it is the same key) or to GitLab with usage type “Signing”.

To verify signatures locally, Git needs an allowed signers file:

```
echo "dana@example.com $(cat ~/.ssh/id_ed25519.pub)" >> ~/.config/git/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.config/git/allowed_signers
git log --show-signature -1
```

```
commit 9f2c1ab4d1e8c3a7f6b0e2d9c5a4b3f2e1d0c9b8
Good "git" signature for dana@example.com with ED25519 key SHA256:Xq3...
Author: Dana Ops <dana@example.com>
```

6.2.2 GPG signing

```
gpg --full-generate-key                  # choose ed25519 or RSA 4096
gpg --list-secret-keys --keyid-format=long
```

```
sec   ed25519/3AA5C34371567BD2 2025-01-10 [SC]
uid                 [ultimate] Dana Ops <dana@example.com>
```

```
git config --global user.signingkey 3AA5C34371567BD2
git config --global commit.gpgsign true
gpg --armor --export 3AA5C34371567BD2    # paste into platform settings
export GPG_TTY=$(tty)                    # needed for passphrase prompts in terminals
```

6.2.3 Verifying

```
git verify-commit HEAD
git verify-tag v1.4.0
git log --pretty="format:%h %G? %GS %s" -5
git merge --verify-signatures feature    # refuse to merge unsigned commits
```

`%G?` prints `G` (good), `B` (bad), `U` (good, unknown validity), `N` (no signature), `E` (cannot check).

Option	Pros	Cons
SSH signing	Reuses existing keys, easy setup, hardware keys (FIDO) work	Needs allowed signers file for local verification
GPG	Mature, web of trust, expiry and revocation	Complex key management
Sigstore gitsign	Keyless, short-lived certs tied to OIDC identity	Newer ecosystem, needs transparency log

> **Interview:** “Does signing a commit prove the code is safe?” No. It proves who created the commit (or at least who held the key). It protects against impersonation and tampering in transit, not against malicious or buggy code. That is what review and CI are for.

6.3 Branching Strategies

A branching strategy is an agreement about which branches exist, how long they live, how code moves between them and how releases are cut. The right choice depends on how you release, not on fashion.

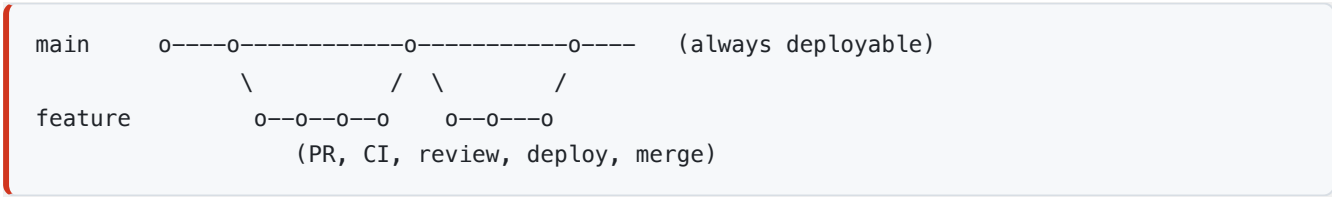
6.3.1 GitFlow



- `main`: production releases, every commit tagged.
- `develop`: integration branch for the next release.
- `feature/*`: off develop, merged back to develop.
- `release/*`: off develop to stabilize, merged to main and develop.
- `hotfix/*`: off main, merged to main and develop.

Fits: software shipped in versions on a schedule, multiple supported versions, mobile apps, on-prem products. Costs: long-lived branches, merge overhead, delayed integration, and it is a poor fit for continuous deployment. Even its original author later recommended simpler flows for web applications.

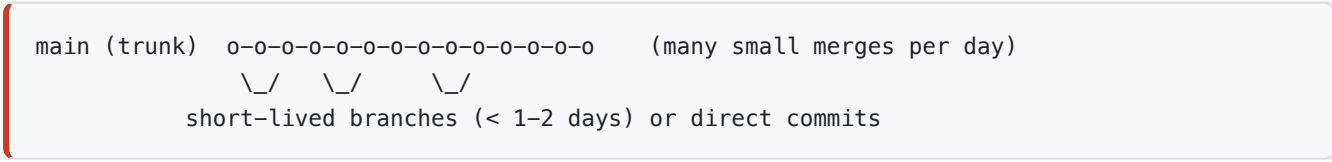
6.3.2 GitHub Flow



1. Branch off `main`.
2. Commit and push; open a pull request early.
3. CI runs; teammates review.
4. Deploy (to staging or even production from the branch) and verify.
5. Merge to `main`, which is always deployable.

Fits: web services and SaaS with continuous delivery, small to medium teams. Costs: needs good automated tests; no built-in notion of maintaining old versions.

6.3.3 Trunk-based development



- Everyone integrates into `main` at least daily.
- Branches, if used, live hours, not weeks.
- Incomplete features are hidden behind **feature flags**, not branches.

- Releases are cut from main (by tag or a short-lived release branch).

Fits: high-performing teams practising continuous integration in its literal sense; the DORA research associates it with better delivery performance. Costs: requires strong CI, fast tests, feature flag discipline and a culture of small changes.

6.3.4 Release branches

Often combined with GitHub Flow or trunk-based development when you must support versions in parallel:

```
main          o--o--o--o--o--o--o--o--o--o
                    \           \
release/2.3          o--x' v2.3.1  \           (x' = cherry-picked fix)
                                    \
release/2.4                          o--y' v2.4.1
```

Rule: fix on `main` first, then cherry-pick (`-x`) to supported release branches. This guarantees no fix is lost in a future release. Fix directly on the release branch only when the code no longer exists on main, and forward-port explicitly.

6.3.5 Environment branches (and why to avoid them)

Some teams keep `dev`, `staging` and `prod` branches and promote by merging between them. This tends to drift: branches diverge, merges conflict, and "what is in prod?" becomes a diff exercise. Prefer one main branch plus promotion of **immutable artifacts** (a container digest or a tag) through environments. In GitOps (Part 7), environments are best modelled as directories or overlays, not branches.

6.3.6 Choosing

Situation	Recommended
SaaS, deploy many times a day, strong CI	Trunk-based with feature flags
Web app, deploy on merge, moderate CI maturity	GitHub Flow
Versioned product, multiple supported releases	GitHub Flow or trunk + release branches
Scheduled releases with long QA cycles, regulated	GitFlow (or a simplified variant)
Infrastructure as code repo	Trunk-based/GitHub Flow; plan on PR, apply on merge
GitOps config repo	Single main branch, environments as directories

6.4 Pull Requests and Code Review

A pull request (GitHub, Bitbucket) or merge request (GitLab) is a proposal to merge one branch into another, with discussion, review and automated checks attached.

6.4.1 What makes a good PR

- Small: ideally under 400 changed lines. Review quality drops sharply with size.
- One purpose, stated in the title (often a Conventional Commit, since squash merges use it).
- A description answering: what, why, how to test, risk and rollback plan.
- Linked ticket, screenshots or `terraform plan` output where relevant.
- Draft PRs for early feedback; mark ready when CI is green.

A PR template enforces this:

```
<!-- .github/pull_request_template.md -->
## What
## Why
## How to test
## Risk and rollback
- [ ] Backward compatible
- [ ] Runbook/docs updated
```

6.4.2 Reviewing well

Focus first	Focus later or automate
Correctness and edge cases	Formatting (formatters)
Security: secrets, permissions, injection	Lint style (linters)
Operability: logging, metrics, rollback	Typos in comments
Blast radius: IAM, network, data deletion	Naming bikesheds

For infrastructure changes, review the plan, not just the code: a two-line Terraform diff can destroy a database.

6.4.3 Merge methods

Method	Result on main	Pros	Cons
Merge commit	All branch commits plus a merge commit	Full history, easy whole-PR revert (`-m 1`)	Noisy graph
Squash and merge	One commit per PR	Clean, linear, one revert per PR	Loses individual commits; branch looks unmerged
Rebase and merge	Branch commits replayed linearly, no merge commit	Linear and keeps commits	New hashes; requires disciplined commits

Most teams pick one and disable the others to keep history consistent. Squash merge is the most common default because it tolerates messy branch history.

6.4.4 Merge queues

On busy repositories, two PRs can each pass CI against an old main and break main when combined. A merge queue (GitHub merge queue, GitLab merge trains, Bitbucket via apps) tests each PR on top of main plus the PRs ahead of it, and merges only if that combination passes.

6.5 Branch Protection and Rulesets

Protect `main` and release branches so that policy is enforced by the server, not by goodwill.

Rule	Why
Require pull request before merging	No direct pushes to main
Require N approvals	Four-eyes principle, audit compliance
Dismiss stale approvals on new commits	Approval applies to what is merged
Require review from code owners	Right experts approve their areas
Require status checks to pass (and branch up to date)	CI must be green on current main
Require signed commits	Identity verification
Require linear history	Enforces squash or rebase merges
Require conversation resolution	No ignored review comments
Block force pushes and deletions	Protects history
Restrict who can push / bypass	Limit emergency access, audit bypasses
Tag protection (`v*`)	Only release automation can create release tags

GitHub **rulesets** extend classic branch protection: multiple rulesets can layer, they apply to branches or tags by pattern across many repositories at the organization level, and they support “evaluate” mode to test before enforcing. GitLab offers protected branches, protected tags, push rules (commit message regex, file size, secret-like file names) and approval rules. Bitbucket has branch permissions and merge checks.

6.6 CODEOWNERS

A `CODEOWNERS` file maps paths to the people or teams responsible. Platforms auto-request their review, and branch protection can require their approval.

```
# .github/CODEOWNERS   (GitHub: .github/, root, or docs/; GitLab: same, plus sections)

# Default owners for everything
*                       @acme/platform-team

# Terraform: network changes need the network team
```

```
/terraform/network/     @acme/network-eng
/terraform/iam/         @acme/security @acme/platform-team

# Helm charts
/charts/                @acme/sre

# CI definitions are security-sensitive
/.github/workflows/     @acme/platform-security
/.gitlab-ci.yml         @acme/platform-security

# Protect the ownership file itself
/.github/CODEOWNERS     @acme/platform-leads
```

Rules: later matching lines take precedence (GitHub), so put general rules first and specific ones after. Owners must have write access to the repository. Protect `CODEOWNERS` itself, otherwise anyone can remove themselves from review requirements in their own PR.

GitLab adds sections and optional approval counts:

```
[Terraform][2] @acme/network-eng @acme/platform-team
/terraform/
```

6.7 Platform Specifics

Concept	GitHub	GitLab	Bitbucket
Change proposal	Pull request	Merge request	Pull request
CI	GitHub Actions (`.github/workflows/*.yml`)	GitLab CI (`.gitlab-ci.yml`)	Bitbucket Pipelines (`bitbucket-pipelines.yml`)
Branch rules	Branch protection, rulesets	Protected branches, push rules, approval rules	Branch permissions, merge checks
Ownership	`CODEOWNERS`	`CODEOWNERS` with sections	Code owners (Cloud) / reviewers add-ons
Merge serialization	Merge queue	Merge trains	Via apps
Secret scanning	Secret scanning, push protection	Secret detection, push protection	Secret scanning (Cloud)
Server refs for reviews	`refs/pull/N/head`, `refs/pull/N/merge`	`refs/merge-requests/N/head`	`refs/pull-requests/N/from` (Server)
Auto-close issues	`Fixes #123`	`Closes #123`	`Fixes PROJ-123` with Jira

Concept	GitHub	GitLab	Bitbucket
CLI	`gh`	`glab`	(REST API, Atlassian CLI)

Command line examples:

```
gh pr create --fill --base main --draft
gh pr checkout 123
gh pr view --web
gh pr merge 123 --squash --delete-branch
glab mr create --fill --target-branch main
glab mr checkout 45
```

GitLab also supports **push options** to create an MR directly from a push:

```
git push -o merge_request.create -o merge_request.target=main -o
merge_request.remove_source_branch
```

6.8 Lab: Enforce Quality Before It Reaches Main

Goal: install hooks with the pre-commit framework, add a custom commit-msg hook, sign commits with SSH, and draft branch protection and CODEOWNERS.

1. Create a repository and configure pre-commit:

`bash cd /tmp && rm -rf lab6 && mkdir lab6 && cd lab6 && git init pipx install pre-commit || pip install --user pre-commit`

Write `.pre-commit-config.yaml` with `trailing-whitespace`, `check-yaml`, `detect-private-key` and `check-added-large-files` (section 6.1.3), then:

`bash pre-commit install git add .pre-commit-config.yaml && git commit -m "ci: add pre-commit"`

2. Trigger a failure:

`bash printf -- "-----BEGIN OPENSSH PRIVATE KEY-----\nfake\n-----END OPENSSH PRIVATE KEY-----\n" > id_test git add id_test && git commit -m "feat: add key" # blocked git rm --cached id_test && rm id_test`

3. Add the commit-msg hook from section 6.1.1 in `.githooks/`, make it executable, set `git config core.hooksPath .githooks` and observe that a message like “updated stuff” is rejected while “docs: add readme” passes.

> **Note:** `core.hooksPath` replaces `.git/hooks`, so pre-commit's installed hook is no longer used. Either install pre-commit into the same directory or keep only one mechanism; in practice teams choose pre-commit and add commit-msg checks to its config.

4. Configure SSH signing (section 6.2.1), commit, then run `git log --show-signature -1` and `git log --pretty="%h %G? %s"`.
5. Write a `CODEOWNERS` file assigning `/terraform/` and `/.github/` to different teams, protecting `CODEOWNERS` itself.
6. On a GitHub or GitLab sandbox repository, protect `main`: require a PR, one approval, code owner review, passing status check, and block force pushes. Try to `git push origin main` directly and record the error.
7. Write one paragraph recommending a branching strategy for a team that deploys a SaaS API ten times a day but also ships a versioned CLI quarterly.

6.9 Key Takeaways

- Hooks automate checks at commit and push time; share them via `core.hooksPath` or a manager like pre-commit.
- Client-side hooks can be bypassed; enforce critical rules in CI and on the server too.
- Sign commits and tags (SSH signing is the easiest path) and require signatures on protected branches where identity matters.
- Choose the branching strategy from your release model: trunk-based or GitHub Flow for continuous delivery, release branches for supported versions, GitFlow only for scheduled versioned releases.
- Keep PRs small and well described; standardize on one merge method; use merge queues on busy repos.
- Protect main and release branches with required reviews, status checks, code owners and no force pushes.
- CODEOWNERS routes reviews to the right experts; protect the file itself.
- Environments should be promotion of immutable artifacts or directories, not long-lived environment branches.

Part 7 — Git in CI/CD, GitOps and Secrets Security

For a DevOps engineer, Git is not just where code lives; it is the event source for automation. A push builds, a pull request tests and plans, a tag releases, and a merged commit in a config repository changes production. This Part covers how pipelines are triggered by Git events, how CI checks out code efficiently, how GitOps controllers reconcile clusters from Git, and how to prevent and clean up leaked secrets.

7.1 Git Events as Pipeline Triggers

Git event	Typical pipeline
Push to feature branch	Lint, unit tests, build
Pull/merge request opened or updated	Full tests, security scans, `terraform plan`, preview environment
Merge to `main`	Build artifact, deploy to staging, publish `latest`
Push tag `v*.*.*`	Build release artifacts, sign, publish, deploy to production
Push to `release/*`	Build release candidates, run long regression suites
Changes under a path	Build only the affected service (monorepo)
Schedule (cron)	Nightly scans, dependency updates

The principle: **the commit is the unit of deployment, and the tag is the unit of release.** Every artifact should be traceable to the exact commit hash it was built from.

7.2 GitHub Actions

7.2.1 Triggers by branch, tag and path

```yaml
# .github/workflows/ci.yml
name: ci

on:
  push:
    branches: [main, "release/**"]
    tags: ["v*.*.*"]
    paths-ignore: ["docs/**", "**.md"]
  pull_request:
    branches: [main]
    paths: ["services/api/**", ".github/workflows/ci.yml"]
  workflow_dispatch: {}
```

```yaml
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make test

  release:
    if: startsWith(github.ref, 'refs/tags/v')
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: write
      packages: write
      id-token: write
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0          # full history so changelog and describe work
      - name: Build image
        run: |
          VERSION="${GITHUB_REF_NAME#v}"
          docker build -t ghcr.io/acme/api:"$VERSION" \
            --label org.opencontainers.image.revision="$GITHUB_SHA" .
```

Key context values:

Variable	Example	Meaning
`GITHUB_SHA`	`9f2c1ab...`	Commit that triggered the run
`GITHUB_REF`	`refs/heads/main`, `refs/tags/v1.4.0`, `refs/pull/88/merge`	Full ref
`GITHUB_REF_NAME`	`main`, `v1.4.0`, `88/merge`	Short name
`GITHUB_HEAD_REF`	`feature/pdb`	PR source branch (PR events only)
`GITHUB_BASE_REF`	`main`	PR target branch

> **Note:** For `pull_request` events, `actions/checkout` checks out a **merge commit** (`refs/pull/N/merge`) of the PR into the base branch, not the PR head. CI therefore tests what main would look like after merging. `GITHUB_SHA` is that merge commit; the PR head is `github.event.pull_request.head.sha` .

> **Warning:** `pull_request_target` runs with the base repository's secrets and write token. Never check out and execute code from a fork's PR in a `pull_request_target` workflow; that is a well-known path to secret theft.

7.2.2 Shallow clones in CI

`actions/checkout` defaults to `fetch-depth: 1` : only the triggering commit, no history. This is fast, but breaks anything that needs history.

Need	Setting
Build and test only	`fetch-depth: 1` (default)
`git describe` , changelog, semantic-release	`fetch-depth: 0` (full history and tags)
Diff against the PR base	`fetch-depth: 0` , or fetch the base explicitly
Large repo, need history but not file contents	`filter: blob:none` with `fetch-depth: 0`
Monorepo, only some directories	`sparse-checkout: services/api`
Tags only	`fetch-tags: true`

Computing changed files in a PR with a shallow clone:

```
- uses: actions/checkout@v4
  with:
    fetch-depth: 0
    filter: blob:none
- name: Changed files
  run: git diff --name-only "origin/${GITHUB_BASE_REF}...HEAD"
```

Common shallow-clone errors and causes:

```
fatal: No names found, cannot describe anything.
```

No tags were fetched: use `fetch-depth: 0` or `git fetch --tags --unshallow` .

```
fatal: origin/main...HEAD: no merge base
```

History too shallow to find the merge base: deepen with `git fetch --deepen=100 origin main` or fetch full history.

7.2.3 Pushing back from CI

Workflows that bump versions or update manifests need to commit:

```
- name: Commit image tag bump
  run: |
    git config user.name  "github-actions[bot]"
    git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
    yq -i '.image.tag = "${{ github.sha }}"' deploy/api/values.yaml
    git commit -am "chore(deploy): api ${{ github.sha }}"
    git push
```

Pushes made with the default `GITHUB_TOKEN` do not trigger further workflows (to avoid infinite loops). Use a GitHub App token when a downstream workflow must run, and add `[skip ci]` to the message when it must not.

7.3 GitLab CI

7.3.1 Rules by branch, tag, MR and path

```
# .gitlab-ci.yml
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never                    # avoid duplicate branch + MR pipelines
    - if: $CI_COMMIT_BRANCH
    - if: $CI_COMMIT_TAG

variables:
  GIT_DEPTH: "20"
  GIT_STRATEGY: fetch

stages: [test, build, release]

test:
  stage: test
  script: make test
  rules:
    - changes:
        - services/api/**/*
        - .gitlab-ci.yml

plan:
  stage: test
```

```
  script: terraform -chdir=terraform plan -out=plan.tfplan
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes: [terraform/**/*]

build:
  stage: build
  script:
    - docker build -t "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA" .
    - docker push "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

release:
  stage: release
  variables:
    GIT_DEPTH: "0"                    # full history for changelog
  script:
    - docker build -t "$CI_REGISTRY_IMAGE:$CI_COMMIT_TAG" .
    - docker push "$CI_REGISTRY_IMAGE:$CI_COMMIT_TAG"
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
```

Variable	Meaning
`CI_COMMIT_SHA` / `CI_COMMIT_SHORT_SHA`	Commit being built
`CI_COMMIT_BRANCH`	Branch name (not set for tag or MR pipelines)
`CI_COMMIT_TAG`	Tag name (only for tag pipelines)
`CI_DEFAULT_BRANCH`	Usually `main`
`CI_PIPELINE_SOURCE`	`push`, `merge_request_event`, `schedule`, `web`…
`CI_MERGE_REQUEST_TARGET_BRANCH_NAME`	MR target
`CI_MERGE_REQUEST_DIFF_BASE_SHA`	Merge base, useful for diffs

7.3.2 Git strategy and depth

Setting	Effect
`GIT_STRATEGY: fetch`	Reuse the runner's existing working copy (faster)
`GIT_STRATEGY: clone`	Fresh clone every job (clean, slower)
`GIT_STRATEGY: none`	No checkout (jobs that only use artifacts)
`GIT_DEPTH: "20"`	Shallow clone depth (project default is often 20 or 50)
`GIT_DEPTH: "0"`	Full history
`GIT_SUBMODULE_STRATEGY: recursive`	Check out submodules

Setting	Effect
`GIT_CLEAN_FLAGS: -ffdx`	How the working copy is cleaned with `fetch`
`GIT_LFS_SKIP_SMUDGE: "1"`	Skip downloading LFS objects

7.4 CI Checkout Best Practices

- **Pin by commit, build once.** Build an artifact once per commit, tag it with the commit SHA, and promote that same artifact across environments.
- **Label artifacts with provenance.** Add the commit SHA and repository URL as OCI labels or build metadata; sign with cosign and produce SLSA provenance where possible.
- **Use the shallowest checkout that works.** Default to depth 1, opt in to more history only for jobs that need it.
- **Fetch tags deliberately.** Release jobs need tags; most jobs do not.
- **Cache dependencies, not `.git`.** Caching Git directories across jobs is fragile; runners with `GIT_STRATEGY: fetch` already reuse them.
- **Treat CI tokens as least privilege.** Read-only by default (`permissions: contents: read`), write only in release jobs, prefer OIDC to long-lived cloud keys.
- **Mark the workspace safe in containers.** When the repository is owned by a different UID than the container user, Git refuses with "detected dubious ownership"; fix with `git config --global --add safe.directory "$PWD"` rather than disabling the check globally.

7.5 GitOps

GitOps applies Git workflows to operations. The OpenGitOps principles state that the desired state of a system is:

1. **Declarative**: described as data (Kubernetes manifests, Helm values, Kustomize overlays).
2. **Versioned and immutable**: stored in Git, with full history.
3. **Pulled automatically**: software agents pull the desired state from the source.
4. **Continuously reconciled**: agents observe actual state and correct drift.

```
Developer --PR--> app repo --CI--> image registry (api:1.4.0, digest sha256:...)
                                          |
                    CI or bot opens PR    v
Reviewer --approve--> config repo (deploy/prod/api/values.yaml: tag 1.4.0)
                                          |
                          pull (poll or webhook)
                                          v
                     Argo CD / Flux in the cluster
                                          |
                           apply and reconcile
                                          v
                              Kubernetes cluster
```

Why it matters:

- **Audit trail**: every production change is a reviewed commit with an author and approver.
- **Rollback**: `git revert` the change; the controller converges back.
- **Drift correction**: manual `kubectl edit` changes are detected and reverted (or flagged).
- **Security**: CI no longer needs cluster credentials; the controller inside the cluster pulls.
- **Disaster recovery**: rebuild a cluster by pointing a controller at the repo.

7.5.1 Repository layout

Separate application source from deployment configuration. App code changes many times a day; config changes are deployments and deserve their own review and permissions.

```
platform-config/
  apps/
    api/
      base/                 # common Kustomize base or Helm values
        deployment.yaml
        kustomization.yaml
      overlays/
        dev/kustomization.yaml
        staging/kustomization.yaml
        prod/kustomization.yaml
  clusters/
    prod-eu-1/
      apps.yaml             # which apps run on this cluster
    staging-eu-1/
      apps.yaml
```

> **Tip:** Model environments as directories on one `main` branch, not as branches. Promotion becomes a small, reviewable PR that copies a version from the staging overlay to the prod overlay, and `main` always describes every environment at once.

7.5.2 Argo CD

Argo CD runs in the cluster and syncs **Applications**, each pointing at a Git path and a target revision.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: api-prod
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/acme/platform-config.git
    targetRevision: main          # branch, tag or commit SHA
    path: apps/api/overlays/prod
```

```
  destination:
    server: https://kubernetes.default.svc
    namespace: api
  syncPolicy:
    automated:
      prune: true                  # delete resources removed from Git
      selfHeal: true               # revert manual drift
    syncOptions: [CreateNamespace=true]
```

Concepts: **sync status** (does the cluster match Git?), **health status** (are resources healthy?), **ApplicationSets** (generate many Applications from a template per cluster or directory), and the **app-of-apps** pattern.

7.5.3 Flux

Flux is a set of controllers. A `GitRepository` source defines what to pull; a `Kustomization` (or `HelmRelease`) defines what to apply from it.

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: platform-config
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/acme/platform-config.git
  ref:
    branch: main
  secretRef:
    name: platform-config-auth
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: api-prod
  namespace: flux-system
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: platform-config
  path: ./apps/api/overlays/prod
  prune: true
  wait: true
```

Flux can also verify commit signatures on the source (`spec.verify`) and has image automation controllers that commit new image tags back to Git.

Aspect	Argo CD	Flux
UI	Rich built-in web UI	CLI-first; UIs via third parties
Model	Application CRDs, projects, RBAC	Composable controllers (source, kustomize, helm, notification, image)
Multi-cluster	Central Argo manages many clusters	Typically one Flux per cluster, bootstrapped from Git
Image updates	Argo CD Image Updater (separate)	Built-in image automation controllers

7.5.4 Git practices for GitOps repos

- Protect `main` heavily; it is effectively production access.
- Require CODEOWNERS approval for `overlays/prod/`.
- Pin images by digest or immutable tag, never `latest`.
- Never put plain secrets in the repo; use Sealed Secrets, SOPS with KMS/age, or External Secrets Operator referencing a vault.
- Keep bot commits (image bumps) identifiable and signed.
- Roll back with `git revert`, not by editing the cluster.

7.6 Secrets: Prevention

Once a secret is pushed, assume it is compromised. Public GitHub repositories are scanned by attackers within minutes, and private repositories get cloned to laptops, CI caches and forks. Prevention is far cheaper than cleanup.

Layered defence:

Layer	Control
Design	Secrets in a vault or cloud secret manager; workloads use OIDC/IAM roles, not static keys
Repo hygiene	`.gitignore` for `.env`, `*.pem`, `*.tfvars`, kubeconfigs; commit `.env.example` only
Developer machine	pre-commit with gitleaks or detect-secrets
Push time	Platform push protection (GitHub secret scanning push protection, GitLab secret push protection)
CI	gitleaks or trufflehog on every PR and on full history periodically
Monitoring	Platform secret scanning alerts, partner programs that auto-revoke known token formats

7.6.1 gitleaks

```
gitleaks git -v                       # scan full history of the repo in the current dir
gitleaks git --log-opts="origin/main..HEAD"   # only new commits (PR scan)
gitleaks dir .                        # scan files on disk, not history
gitleaks git --report-format sarif --report-path gitleaks.sarif
```

> **Note:** Older gitleaks versions (before 8.19) used `gitleaks detect` for history and `gitleaks protect --staged` for staged changes. Both still appear in many tutorials and CI templates.

```
Finding:     aws_secret_access_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
Secret:      wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
RuleID:      aws-secret-access-key
Entropy:     4.7
File:        terraform/provider.tf
Line:        7
Commit:      b13a9e5c2f...
Author:      Lee Chen
Date:        2025-05-19T16:02:51Z
```

Allowlist false positives in `.gitleaks.toml` or with a `gitleaks:allow` comment, and keep a baseline for legacy findings so CI fails only on new leaks.

In GitHub Actions:

```yaml
- uses: actions/checkout@v4
  with:
    fetch-depth: 0
- uses: gitleaks/gitleaks-action@v2
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

7.7 Secrets: Response and History Cleanup

If a secret reaches a remote, follow this order. The order matters.

1. **Revoke and rotate the secret immediately.** This is the only step that actually removes the risk. Cleaning history does not un-leak a credential.
2. **Investigate use.** Check cloud audit logs (CloudTrail, Azure Activity Log, GCP Audit Logs) for activity with that credential since it was pushed.
3. **Remove it from the current code** with a normal commit, and move it into a secret manager.
4. **Decide whether to rewrite history.** Required for compliance or when the secret cannot be rotated (rare). Rewriting is disruptive: every clone, fork and open PR is affected.
5. **Rewrite, force push, and coordinate** with everyone who has a clone.

6. **Purge caches** on the platform (GitHub support can remove cached views and PR refs; GitLab offers repository cleanup and housekeeping).

7.7.1 git filter-repo

`git filter-repo` is the recommended tool for history rewriting; `git filter-branch` is deprecated and slow. Install with `pip install git-filter-repo` or `brew install git-filter-repo`.

It insists on a fresh clone so you cannot destroy your only copy:

```
git clone --mirror git@github.com:acme/platform-infra.git
cd platform-infra.git
```

Remove a file from all history:

```
git filter-repo --invert-paths --path terraform/prod.tfvars
```

Remove all files matching a glob:

```
git filter-repo --invert-paths --path-glob '*.pem'
```

Replace secret strings everywhere in history:

```
cat > ../replacements.txt <<'EOF'
wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY==>***REMOVED***
regex:AKIA[0-9A-Z]{16}==>AKIA_REMOVED
EOF
git filter-repo --replace-text ../replacements.txt
```

Remove large blobs:

```
git filter-repo --strip-blobs-bigger-than 50M
```

Then push the rewritten history:

```
git remote add origin git@github.com:acme/platform-infra.git   # filter-repo removes origin
as a safety measure
git push --force --mirror origin
```

> **Warning:** `--mirror` force-pushes every ref, including deleting branches that do not exist locally. Make sure branch protection is temporarily relaxed only for the person doing the cleanup, and that everyone has been told to stop pushing. Re-enable protection immediately afterwards.

Afterwards, every collaborator must discard their old clone and re-clone (or hard reset every branch). An old clone pushed later will reintroduce the secret. Open PRs that reference old commits must be recreated.

Tool	Notes
`git filter-repo`	Recommended, fast, flexible, maintained
BFG Repo-Cleaner	Java, simple for deleting files and replacing text; protects HEAD by default
`git filter-branch`	Deprecated; slow and easy to misuse

7.8 Lab: Pipeline, GitOps and a Leaked Key

Goal: wire Git events to CI, observe shallow clone limitations, model a GitOps repo, and run the full secret-leak response.

1. **Shallow clone limits.** Using a repo with tags (Lab 4 or any public project):

```
bash git clone --depth 1 file:///tmp/lab4 /tmp/lab7-shallow cd /tmp/lab7-shallow git describe --tags # fails git fetch --unshallow --tags git describe --tags # works
```

Note: `file://` is required; a plain path ignores `--depth`.

2. **CI workflow.** Write a GitHub Actions or GitLab CI file that runs tests on PRs, builds on `main`, and releases only on tags matching `v*.*.*`, with `fetch-depth: 0` (or `GIT_DEPTH: "0"`) only in the release job. If you have a sandbox repo, push a branch, open a PR, then push a tag and confirm the right jobs run.

3. **GitOps layout.** Create a `platform-config` repository with `apps/api/base` and `apps/api/overlays/{staging,prod}`. Write an Argo CD `Application` or Flux `Kustomization` for prod. Simulate a promotion as a PR that changes the image tag in `overlays/prod`, then simulate a rollback with `git revert`.

4. **Leak and detect:**

```
bash cd /tmp && rm -rf lab7 && mkdir lab7 && cd lab7 && git init echo 'provider "aws" { region = "eu-west-1" }' > main.tf git add . && git commit -m "feat: provider" echo 'aws_secret_access_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"' > creds.tf git add . && git commit -m "feat: creds" git rm creds.tf && git commit -m "fix: remove creds" git log --oneline -S "wJalrX" --all # still in history gitleaks git -v # if gitleaks is installed
```

5. **Clean history** (pretend you have already rotated the key):

```
bash cd /tmp && git clone --no-local lab7 lab7-clean && cd lab7-clean git filter-repo --invert-paths --path creds.tf git log --oneline --all -S "wJalrX" # nothing git log --oneline # note the changed hashes
```

6. Write a short incident note: when the key was pushed, when it was rotated, what audit logs showed, and who must re-clone.

7.9 Key Takeaways

- Branch pushes, PRs, merges to main and tag pushes map naturally to test, plan, deploy and release pipelines.
- Build once per commit, label artifacts with the commit SHA, and promote the same artifact through environments.
- CI defaults to shallow clones; request full history (`fetch-depth: 0`, `GIT_DEPTH: "0"`) only for jobs that need tags or merge bases.
- PR pipelines on GitHub test a merge commit of the PR into the base branch; never run untrusted fork code with secrets.
- GitOps: declarative, versioned, pulled and continuously reconciled; Argo CD and Flux sync clusters from Git, and `git revert` is your rollback.
- Model environments as directories in a config repo, not branches.
- Prevent secret leaks in layers: design, ignore files, pre-commit, push protection, CI scanning.
- If a secret leaks: rotate first, investigate, then optionally rewrite history with `git filter-repo`, force push and have everyone re-clone.

Part 8 — Troubleshooting Common Disasters

Sooner or later you will be the person someone messages with “I think I broke Git”. This Part is a field guide. Each scenario lists the symptom, why it happened, and the fix. The general method is always the same:

1. **Stop.** Do not run more commands hoping something works; each one can overwrite evidence.
2. **Look.** `git status`, `git log --oneline --graph --all -20`, `git reflog -20`.
3. **Back up.** Create a rescue branch (`git branch rescue-$(date +%s)`) or copy the whole directory, including `.git`, before risky repairs.
4. **Fix** with the smallest reversible step.

> **Tip:** Committed work is almost never truly lost while the reflog lasts (90 days by default). Uncommitted work is the fragile part. That is the strongest argument for committing early and often on local branches.

8.1 Detached HEAD

Symptom:

```
You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.
```

Why: HEAD points directly at a commit instead of at a branch. It happens when you check out a tag, a commit hash, a remote-tracking branch (`git switch --detach origin/main`), or during a rebase or bisect. Commits you make are not on any branch.

Fix: if you made commits you want to keep, give them a branch before switching away:

```
git switch -c fix/from-detached
```

If you already switched away, Git printed the hash:

```
Warning: you are leaving 1 commit behind, not connected to
any of your branches:

  955b38d three
```

```
git branch fix/recovered 955b38d
# or find it later:
```

```
git reflog | grep -m1 "three"
```

8.2 Committed to the Wrong Branch

Symptom: you committed to `main` locally but meant to use a feature branch. Not pushed yet.

```
git branch feature/oops          # new branch at the current commit (keeps the work)
git reset --hard origin/main     # move main back to the remote state
git switch feature/oops
```

If the commits belong on an existing branch:

```
git switch feature/existing
git cherry-pick main~2..main     # copy the last two commits
git switch main
git reset --hard HEAD~2
```

If the commits were already pushed to `main` (and main is protected or shared), use `git revert` on main instead of reset.

8.3 Pushed a Secret

Symptom: a password, token or private key is in a pushed commit.

Why this is urgent: removing it in a new commit leaves it in history; anyone with a clone or fork has it, and automated scanners harvest public repos within minutes.

Fix, in order:

1. Revoke and rotate the credential now. This is the fix; everything else is cleanup.
2. Check audit logs for misuse since the push time.
3. Remove the secret from the code and load it from a secret manager.
4. If policy requires, rewrite history with `git filter-repo --replace-text` or `--invert-paths`, force push all refs, and require every collaborator to re-clone (Part 7).
5. Ask the platform to purge cached views and pull request refs if needed.
6. Add a pre-commit secret scanner and enable push protection so it does not recur.

> **Warning:** Making the repository private after a public leak does not help; the secret was already exposed. Rotation is non-negotiable.

8.4 Someone Force-Pushed Over Work

Symptom: `main` or a shared branch suddenly lost commits after a teammate's `git push --force`.

```
 + 7c3e1a0...a1b2c3d main -> main (forced update)
```

Why: a plain force push replaces the remote branch with the pusher's version, discarding whatever was there.

Fix: the lost commits exist in at least one place: the clone of whoever pushed them, the reflog of anyone who fetched before the force push, or the platform's event log.

```
# On any clone that fetched the old state:
git reflog show origin/main
```

```
a1b2c3d refs/remotes/origin/main@{0}: fetch: forced-update
7c3e1a0 refs/remotes/origin/main@{1}: fetch: fast-forward
```

```
git switch -c restore-main 7c3e1a0
git log --oneline a1b2c3d..7c3e1a0      # the commits that were overwritten
# Decide: restore the old tip entirely, or merge both histories
git switch main
git merge restore-main                  # keep both
git push                                # normal push, no force needed if it fast-forwards
```

If nobody has a local copy, GitHub's activity view and events API record the previous SHA for pushes, and the objects usually remain on the server for a while; GitLab keeps similar push audit events. Contact support quickly, before server-side garbage collection.

Prevent: block force pushes on protected branches, and use `--force-with-lease --force-if-includes` on personal branches.

8.5 Merge Gone Wrong

8.5.1 Merge still in progress, conflicts look bad

```
git merge --abort          # back to pre-merge state
git rebase --abort         # same for a rebase
git cherry-pick --abort
```

8.5.2 Merge committed locally, not pushed

```
git reset --hard ORIG_HEAD   # ORIG_HEAD is set before the merge
# or
git reset --hard HEAD~1      # first parent of the merge commit is your previous tip
```

8.5.3 Merge already pushed to a shared branch

Revert the merge, keeping the mainline (first parent):

```
git log --oneline --graph -5
git revert -m 1 4e7d0c2
git push
```

```
      4e7d0c2 (bad merge)
  ---o---M---R  main          R = revert of M, relative to parent 1
        /
   o---o  feature
```

Remember: to merge `feature` again later, first revert the revert (`git revert R`), otherwise Git considers its commits already integrated.

8.5.4 Conflict resolved incorrectly in a merge

Find the merge, inspect what the resolution did compared to each parent:

```
git show --remerge-diff 4e7d0c2   # Git 2.36+: shows how the resolution differs from an
automatic merge
git diff 4e7d0c2^1 4e7d0c2 -- path/file
```

Fix with a normal follow-up commit on the branch; rewriting a pushed merge is rarely worth it.

8.6 Lost Commits After Reset or Rebase

```
git reflog -20
git reset --hard HEAD@{5}        # or the hash from the reflog
```

If the branch itself was deleted:

```
git reflog --all | grep feature/pdb
git branch feature/pdb <sha>
```

Last resort, when no reflog references remain:

```
git fsck --lost-found --no-reflogs
git show <sha-of-dangling-commit>
```

8.7 Lost a Stash

A dropped or cleared stash is an unreachable commit. Find it:

```
git fsck --no-reflogs --unreachable | awk '/commit/ {print $3}' \
  | xargs -n1 git log -1 --format="%h %ci %s" | grep -i "WIP on|On "
git stash apply <sha>
```

8.8 Accidentally Committed a Huge File

Not pushed yet:

```
git rm --cached big.iso
git commit --amend --no-edit      # if it was the last commit
# or for older commits: git rebase -i and edit, or filter-repo on a local clone
```

Pushed: the server will usually reject files over the platform limit (100 MB on GitHub). If it was accepted, remove with `git filter-repo --strip-blobs-bigger-than 50M` or migrate to LFS with `git lfs migrate import --include="*.iso" --everything`, then force push and coordinate.

8.9 “Diverged” Branches and Rejected Pushes

```
hint: You have divergent branches and need to specify how to reconcile them.
fatal: Need to specify how to reconcile divergent branches.
```

Why: both your local branch and the remote have new commits, and `pull` does not know whether you want merge, rebase or fast-forward only.

```
git config --global pull.rebase true    # one-time preference
git pull --rebase                       # or explicitly for this pull
```

8.10 Other Frequent Errors

Error	Cause	Fix
`fatal: refusing to merge unrelated histories`	Two repos with no common commit (e.g. created a README on the platform and also `git init` locally)	`git pull --allow-unrelated-histories` once, if intended
`fatal: detected dubious ownership in repository`	Repo owned by another user (common in containers and CI)	`git config --global --add safe.directory /path/to/repo`
`error: cannot lock ref 'refs/remotes/origin/x'`	Stale or conflicting ref (e.g. `x` and `x/y`)	`git remote prune origin` or `git fetch --prune`
`fatal: index.lock exists`	Crashed or concurrent Git process	Ensure no Git process runs, then `rm .git/index.lock`
`Permission denied (publickey)`	SSH key not loaded or not registered	`ssh-add`, `ssh -T git@github.com`, check `GIT_SSH_COMMAND="ssh -v"`
`remote: Support for password authentication was removed`	Using a password over HTTPS	Use a personal access token, credential manager or SSH
`error: RPC failed; curl 56` / `early EOF`	Large push or flaky network/proxy	Push in smaller batches, check proxy, use SSH, `http.postBuffer` only as last resort
`Your branch is ahead of 'origin/main' by N commits` but push shows nothing	Upstream misconfigured	`git branch -vv`, `git push -u origin HEAD`
`fatal: You are not currently on a branch`	Detached HEAD	`git switch -c new-branch`
Line ending churn, whole file shows changed	CRLF/LF conversion	Add `.gitattributes` with `* text=auto eol=lf`, run `git add --renormalize .`
File mode churn (`old mode 100644 new mode 100755`)	Filesystem without exec bits	`git config core.fileMode false` on that clone
Submodule directory is empty	Not initialized	`git submodule update --init --recursive`

8.11 Lab: Disaster Drills

Goal: practise each recovery until it is routine. Use a bare repo as the remote.

1. Setup:

`bash cd /tmp && rm -rf lab8 && mkdir lab8 && cd lab8 git init --bare origin.git git clone origin.git dev && cd dev for i in 1 2 3; do echo $i > f$i; git add .; git commit -qm "feat: f$i"; done git push -u origin main`

2. **Detached HEAD drill:** `git switch --detach HEAD~1`, commit something, `git switch main`, then recover the commit using the hash Git printed.
3. **Wrong branch drill:** commit to main, move it to `feature/x` with `git branch` plus `git reset --hard origin/main`.
4. **Force-push drill:** in a second clone (`git clone origin.git other`), add a commit and push. In `dev`, without fetching, make a different commit and `git push --force`. Then in `other`, find the lost commit with `git reflog show origin/main` after a fetch (or from `main`'s own reflog) and restore it with a merge.
5. **Bad merge drill:** create `feature/bad`, merge it with `--no-ff`, push, then `git revert -m 1 HEAD` and push. Re-land the feature by reverting the revert.
6. **Lost stash drill:** create a stash, run `git stash clear`, then recover it using `git fsck`.
7. **Rescue reflex:** before each drill, run `git branch rescue-$(date +%s)` and notice how it makes every mistake trivially reversible.

8.12 Key Takeaways

- Stop, look (status, graph, reflog), back up, then fix with the smallest reversible step.
- Detached HEAD is harmless until you leave commits behind; `git switch -c` saves them.
- Pushed secrets require rotation first; history cleanup is secondary and disruptive.
- Force-pushed work survives in other clones' reflogs and platform event logs; restore it with a merge and block force pushes on shared branches.
- Undo unpushed merges with `reset --hard ORIG_HEAD`; undo pushed merges with `revert -m 1`, and remember to revert the revert before re-merging.
- `git reflog` and `git fsck --lost-found` recover almost any committed or stashed work.
- Most cryptic errors (dubious ownership, unrelated histories, index.lock, divergent branches) have short, well-known fixes.

Part 9 — Capstone, Interview Q&A and Cheat Sheet

This final Part ties the course together. The capstone has you design and run a complete team workflow end to end. The interview section collects the questions DevOps candidates are actually asked, with concise model answers. The cheat sheet is a dense reference to keep next to your terminal.

9.1 Capstone Lab: A Production-Grade Team Workflow

Scenario: you are the platform engineer for a team of six building `orders-api`, a containerized service deployed to Kubernetes. The team deploys to production several times a week and must also support the previous minor release for one enterprise customer. You will build the Git workflow, guardrails, pipelines and GitOps promotion, then run a full release cycle including a hotfix and an incident.

Use a GitHub or GitLab sandbox organization if you have one. Where a step needs the platform, the local equivalent is noted.

9.1.1 Design decisions (write them down first)

Record these in `docs/git-workflow.md` in the repository. A written, agreed workflow is the deliverable as much as the code.

Decision	Choice for this scenario	Reason
Branching	Trunk-based with short-lived branches, plus `release/X.Y` branches for supported versions	Frequent deploys, one older version supported
Merge method	Squash merge only	One commit per PR, clean revert
Commit format	Conventional Commits, enforced on PR titles	Automated SemVer and changelog
Versioning	SemVer tags `vX.Y.Z` created by release automation	Immutable releases
Protection	PR required, 1 approval + code owners, CI green, linear history, signed commits, no force push	Compliance and safety
Deployment	GitOps: separate `orders-config` repo, environments as overlays	Audit trail and drift correction
Secrets	External Secrets Operator; gitleaks in pre-commit and CI; push protection on	Nothing secret in Git

9.1.2 Step 1: Create the repositories

```
mkdir -p ~/capstone && cd ~/capstone
git init -b main orders-api && git init -b main orders-config
cd orders-api
mkdir -p src docs .github/workflows
echo "print('orders-api')" > src/app.py
printf ".env\n.env.*\n!.env.example\n__pycache__/\n*.pem\n" > .gitignore
printf "* text=auto eol=lf\n" > .gitattributes
git add . && git commit -m "chore: initial project skeleton"
```

Create both repositories on your platform and push (`git remote add origin ...`, `git push -u origin main`). Locally, `git init --bare` repositories work as remotes.

9.1.3 Step 2: Guardrails

1. Add `.pre-commit-config.yaml` with whitespace, YAML, large-file, private-key and gitleaks hooks (Part 6). Run `pre-commit install` and `pre-commit run --all-files`.
2. Add `CODEOWNERS`:

```
text * @acme/orders-team /.github/ @acme/platform /CODEOWNERS @acme/platform
```

3. Configure SSH commit signing for every team member.
4. Protect `main` and `release/*` with the rules in 9.1.1. Protect tags matching `v*` so only the release bot can create them.
5. Add a PR template with What / Why / How to test / Risk and rollback.

9.1.4 Step 3: Pipelines

Create `.github/workflows/ci.yml` (or `.gitlab-ci.yml`) that:

- On pull requests: runs `pre-commit run --all-files`, unit tests, gitleaks on `origin/main..HEAD`, and validates the PR title against Conventional Commits.
- On push to `main`: builds the image tagged with the commit SHA, pushes it, and opens a PR to `orders-config` bumping the **staging** overlay to that SHA.
- On push of a tag `v*.*.*`: builds the image tagged with the version, signs it, and opens a PR to `orders-config` bumping the **prod** overlay.
- Uses `fetch-depth: 1` everywhere except the release job, which uses `fetch-depth: 0`.

Add a release automation job (release-please or semantic-release) that maintains a release PR with the next version and changelog derived from commits.

9.1.5 Step 4: GitOps configuration

In `orders-config`:

```
apps/orders-api/base/{deployment.yaml,service.yaml,kustomization.yaml}
apps/orders-api/overlays/staging/kustomization.yaml
```

```
apps/orders-api/overlays/prod/kustomization.yaml
clusters/staging/orders-api.yaml     # Argo CD Application or Flux Kustomization
clusters/prod/orders-api.yaml
CODEOWNERS                            # /apps/*/overlays/prod/ @acme/sre-leads
```

Each overlay sets the image with `images: [{name: orders-api, newTag: <sha-or-version>}]`. Protect `main` here as well; this repository is production access.

9.1.6 Step 5: Run a feature through the workflow

```
cd ~/capstone/orders-api
git switch -c feat/ORD-101-idempotency-keys
# implement in two commits
git commit -am "feat(api): accept Idempotency-Key header"
git commit -am "test(api): cover duplicate request handling"
git push
gh pr create --fill --title "feat(api): support idempotency keys"
```

Address review feedback with `git commit --fixup=<sha>`, push, then before merge run `git rebase -i --autosquash origin/main` and `git push --force-with-lease`. Squash merge. Verify that the pipeline built the image and opened the staging promotion PR; merge it and watch the GitOps controller sync.

9.1.7 Step 6: Cut a release and a release branch

Merge the release PR so automation tags `v1.3.0`. Then create the support branch:

```
git fetch --tags
git switch -c release/1.3 v1.3.0
git push -u origin release/1.3
```

Promote `v1.3.0` to prod by merging the config PR. Confirm `git describe --tags` on main shows `v1.3.0-N-g<sha>` as new work lands.

9.1.8 Step 7: Hotfix and backport

A bug is found in production. Fix on main first:

```
git switch -c fix/ORD-140-null-total origin/main
git commit -am "fix(api): handle null order total"
# PR, review, squash merge -> commit abc1234 on main
git switch release/1.3 && git pull
git switch -c backport/ORD-140-to-1.3
git cherry-pick -x abc1234
git push -u origin backport/ORD-140-to-1.3
# PR into release/1.3, merge, tag v1.3.1 from release/1.3
```

9.1.9 Step 8: Incident simulation

Pick two of the following and execute the recovery, writing a short postmortem for each:

1. A developer commits a fake API key and pushes a branch. Show that pre-commit and CI block it; then bypass the hook with `--no-verify` and show CI and push protection still catch it. Run the rotation and history-cleanup procedure on the branch.
2. A bad config change reaches prod. Roll back with `git revert` in `orders-config` and confirm the controller converges.
3. A force push on a feature branch overwrites a teammate's commits. Recover from the teammate's reflog.
4. A regression appeared somewhere in the last 40 commits. Find it with `git bisect run` and a test script.

9.1.10 Capstone deliverables checklist

- `docs/git-workflow.md` with all decisions and their reasons
- Working pre-commit, CODEOWNERS, PR template, branch and tag protection
- CI that tests PRs, builds on main, releases on tags, with correct clone depths
- GitOps repo with environment overlays and a controller definition
- One feature, one release, one backported hotfix, all traceable from commit to running image
- Two postmortems from the incident simulation

9.2 Interview Questions and Answers

9.2.1 Beginner

1. What is Git and how does it differ from GitHub? Git is a distributed version control system that runs locally. GitHub (like GitLab and Bitbucket) is a hosting platform for Git repositories that adds pull requests, CI, permissions and issue tracking.

2. What does "distributed" mean for Git? Every clone contains the full history and can commit, branch and view logs offline. The central server is a convention, and any clone can restore it.

3. Explain the working directory, staging area and HEAD. The working directory holds your editable files. The staging area (index) holds the exact snapshot proposed for the next commit. HEAD points to the current branch and thus the last commit. `add` moves changes into the index; `commit` turns the index into a commit.

4. What is the difference between `git fetch` and `git pull`? `fetch` downloads new commits and updates remote-tracking branches without changing your work. `pull` is `fetch` followed by merging or rebasing the upstream into your current branch.

5. What is the difference between `git diff` and `git diff --staged`? `git diff` compares working directory to index (unstaged changes). `--staged` compares index to HEAD (what will be committed).

6. How do you undo changes to a file you have not staged? `git restore <file>`, which copies the index version back over the working file. The edits are lost permanently.

7. How do you unstage a file? `git restore --staged <file>`. The file content on disk is not changed.

8. What does `.gitignore` do, and why is a file I added to it still tracked? It stops untracked files from being added. It does not affect files already tracked; untrack with `git rm --cached <file>` and commit.

9. What is a branch? A movable pointer to a commit, stored as a small ref file. Committing moves the current branch forward. Creating branches is nearly free.

10. What is the difference between `git switch` and `git checkout`? `checkout` both switches branches and restores files. Git 2.23 split these into `switch` (branches) and `restore` (files) to make commands safer and clearer.

11. What is a fast-forward merge? When the target branch has no new commits since the source branched off, Git simply moves the target pointer forward. No merge commit is created.

12. What makes a good commit message? A short imperative subject (around 50 characters), a blank line, then a body explaining why the change was made. One logical change per commit. Conventional Commits adds a type prefix like `feat:` or `fix:`.

13. What is `origin`? The default name Git gives to the remote you cloned from. It is just a name and can be renamed.

14. What is a tag, and annotated vs lightweight? A tag names a specific commit permanently. Lightweight tags are just refs; annotated tags are objects with tagger, date, message and optional signature. Use annotated tags for releases.

9.2.2 Intermediate

15. Explain a three-way merge. Git finds the merge base (common ancestor) and compares it with both branch tips. Changes made on only one side are applied automatically; conflicting changes to the same region are marked for manual resolution. The result is a merge commit with two parents.

16. How do you resolve a merge conflict? Open each conflicted file, edit to the correct content removing the markers, `git add` the file, then `git merge --continue`. Use `zdiff3` conflict style to see the base, `git log --merge` for context, or `git merge --abort` to back out.

17. Merge vs rebase: when do you use each? Rebase to keep your own unshared branch current and linear, and to clean it up before review. Merge (or squash merge via the platform) to integrate into shared branches. Never rebase shared history.

18. What does `git rebase -i` let you do? Edit local history: reorder, reword, squash, fixup, split (via `edit`), drop commits, or run a command after each commit with `exec`.

19. What are `--fixup` and `--autosquash`? `git commit --fixup=<sha>` creates a commit marked to be folded into `<sha>`. `git rebase -i --autosquash` automatically places and squashes those fixups, which is ideal for addressing review comments.

20. Explain `git reset --soft`, `--mixed` and `--hard`. All move the current branch to a commit. Soft keeps index and working directory (changes become staged). Mixed resets the index but keeps files (changes become unstaged). Hard resets both, discarding uncommitted changes.

21. Reset vs revert? Reset moves the branch pointer and rewrites history; use only on unshared commits. Revert adds a new commit that undoes a previous one; safe on shared branches.

22. How do you revert a merge commit? `git revert -m 1 <merge-sha>`, where `-m 1` says the first parent is the mainline. To re-merge that branch later, revert the revert first.

23. What is cherry-pick used for? Applying a specific commit's change onto another branch, typically backporting a fix from main to a release branch. Use `-x` to record the source commit.

24. What is the reflog? A local log of where HEAD and each branch pointed over time. It lets you recover from bad resets, rebases and deleted branches. Entries expire (90 days by default; 30 for unreachable).

25. What is `--force-with-lease` and why prefer it? It force pushes only if the remote branch is still where your remote-tracking ref says, so you do not overwrite commits someone else pushed meanwhile. Add `--force-if-includes` to guard against background fetches.

26. What is detached HEAD and how do you get out of it? HEAD points directly at a commit rather than a branch (e.g. after checking out a tag). New commits are not on any branch. Run `git switch -c <name>` to keep them, or `git switch <branch>` to leave.

27. How does `git stash` work, and what are its pitfalls? It saves uncommitted changes as special commits on a stash stack and cleans the working tree. `-u` includes untracked files. Pitfalls: stashes are local, easy to forget and can conflict on pop; use `git stash branch` or WIP commits for longer work.

28. What is `A..B` vs `A...B`? In `log`, `A..B` lists commits in B not in A; `A...B` lists commits in either but not both. In `diff`, `A...B` compares the merge base with B, which is what a pull request shows.

29. How would you find which commit introduced a bug? `git bisect start`, mark a bad and a good commit, then test each midpoint or automate with `git bisect run <script>`. It finds the culprit in logarithmic steps.

30. Compare GitFlow, GitHub Flow and trunk-based development. GitFlow uses long-lived `develop` and `main` plus feature, release and hotfix branches; suited to scheduled versioned releases. GitHub Flow uses short-lived branches off an always-deployable main with PRs. Trunk-based integrates into main at least daily with feature flags; best for continuous delivery.

31. What are CODEOWNERS and branch protection for? CODEOWNERS maps paths to responsible reviewers, auto-requests them and can be required for approval. Branch protection enforces rules such as PR reviews, passing checks, signed commits and no force pushes on important branches.

32. Squash merge, rebase merge or merge commit: which would you choose? Squash for clean one-commit-per-PR history and simple reverts; merge commits when individual commits and integration points matter; rebase merge when commits are curated and linear history is required. Pick one per repository for consistency.

9.2.3 Advanced

33. Describe Git's object model. Four object types in a content-addressed store: blobs (file contents), trees (directory listings of mode, type, hash, name), commits (tree, parents, author, committer, message) and annotated tags. Each is named by the hash of its content, forming a Merkle DAG.

34. Why does rebasing change commit hashes? A commit's hash includes its parent hash. Replaying a commit onto a new parent produces a different commit object even with identical changes, and every descendant changes too.

35. What are packfiles and why do they matter? Packfiles bundle many objects with delta compression and an index, drastically reducing disk and transfer size. `git gc` and `git maintenance` create and optimize them; fetch and clone transfer packs.

36. How would you speed up Git on a very large monorepo? Blobless partial clone (`--filter=blob:none`), cone-mode sparse checkout, `scalar`, `core.fsmonitor` and untracked cache, commit-graph and multi-pack index, `git maintenance start`, LFS for binaries, and shallow or treeless clones plus sparse checkout in CI.

37. Shallow clone vs partial clone? Shallow truncates history (depth), breaking history-dependent commands. Partial keeps full history but defers downloading blobs or trees until needed, so history commands still work.

38. Submodules vs subtrees? Submodules store a pointer to an exact commit of another repo; precise and small but require extra clone and update steps. Subtrees copy files into a subdirectory; transparent for consumers but heavier and awkward to push upstream.

39. A secret was pushed to a public repository. Walk me through the response. Rotate the credential immediately, check audit logs for misuse, remove it from code and move it into a secret manager, then if required rewrite history with `git filter-repo`, force push all refs, have everyone re-clone, ask the platform to purge caches, and add pre-commit scanning plus push protection.

40. How do you prevent secrets from being committed? Design for no static secrets (OIDC, workload identity, vaults), ignore files, pre-commit hooks such as gitleaks, platform push protection, CI scanning on every PR and periodic full-history scans.

41. What is GitOps and how do Argo CD and Flux implement it? Desired state is declared in Git, versioned, pulled by agents and continuously reconciled. Argo CD uses Application resources pointing at repo paths and revisions with auto-sync and self-heal. Flux uses GitRepository sources and Kustomization or HelmRelease controllers. Rollback is a Git revert.

42. Why avoid environment branches in GitOps? Branches per environment drift apart, cause merge conflicts and hide what is deployed where. Directories or overlays on one main branch make promotion an explicit, reviewable diff and show all environments at once.

43. Your CI job fails with "No names found, cannot describe anything." Why? The checkout is shallow and has no tags. Fetch full history and tags (`fetch-depth: 0`, `GIT_DEPTH: "0"`, or `git fetch --tags --unshallow`).

44. A teammate force-pushed over main. How do you recover? Find the previous tip from any clone's reflog (`git reflog show origin/main`) or platform push events, create a branch there, compare with the current tip, merge or restore, and push. Then block force pushes on main.

45. How do Git hooks fit into an enforcement strategy? Client hooks (pre-commit, commit-msg, pre-push) give fast feedback but can be skipped with `--no-verify`. Enforcement must be repeated in CI and server-side controls: pre-receive hooks, branch protection, push rules and required status checks.

46. How do commit signatures work, and what do they prove? The committer signs the commit object with a GPG, SSH or X.509 key; the platform verifies against registered keys and marks it Verified. It proves the holder of the key created the commit, not that the code is correct or safe.

47. What is `rerere`? "Reuse recorded resolution". Git records how you resolved a conflict and automatically reapplies the same resolution when the identical conflict reappears, which helps with repeated rebases and long-running branches.

48. How would you design versioning and release automation for a service? Conventional Commits enforced on PR titles, squash merges, a release tool (release-please or semantic-release) computing SemVer from commits, annotated signed tags, tag-triggered pipelines building immutable artifacts labelled with the commit SHA, and GitOps promotion of the tagged artifact.

9.3 Command Cheat Sheet

9.3.1 Setup and configuration

Command	Purpose
`git config --global user.name "Name"`	Set author name
`git config --global user.email "a@b.c"`	Set author email
`git config --global init.defaultBranch main`	Default branch name
`git config --global pull.rebase true`	Rebase on pull
`git config --global push.autoSetupRemote true`	Auto upstream on first push
`git config --global merge.conflictStyle zdiff3`	Show base in conflicts
`git config --global rerere.enabled true`	Reuse conflict resolutions
`git config --global core.excludesFile ~/.config/git/ignore`	Global ignore
`git config --list --show-origin`	Show all config and sources

9.3.2 Create and inspect

Command	Purpose
`git init -b main`	New repository

Command	Purpose
`git clone <url>`	Clone
`git clone --depth 1 <url>`	Shallow clone
`git clone --filter=blob:none <url>`	Blobless partial clone
`git status -sb`	Short status with branch
`git diff` / `--staged` / `HEAD`	Unstaged / staged / all changes
`git diff main...feature --stat`	What a branch introduces
`git log --oneline --graph --all`	Visual history
`git log -S "text"`	Commits adding or removing text
`git log --follow -- file`	File history across renames
`git show <rev>:<path>`	File at a revision
`git blame -w -C -L 10,20 file`	Line authorship

9.3.3 Stage and commit

Command	Purpose
`git add -p`	Stage hunks interactively
`git restore <file>`	Discard unstaged changes
`git restore --staged <file>`	Unstage
`git restore --source=<rev> <file>`	File from another commit
`git commit -m "type(scope): msg"`	Commit
`git commit --amend --no-edit`	Add to last commit
`git commit --fixup=<sha>`	Fixup for autosquash
`git rm --cached <file>`	Untrack, keep on disk
`git clean -nd` then `-fd`	Preview / delete untracked

9.3.4 Branch and merge

Command	Purpose
`git switch -c <branch> [start]`	Create and switch
`git switch -`	Previous branch
`git branch -vv`	Branches with upstream

Command	Purpose
`git branch -d / -D <branch>`	Delete / force delete
`git branch --merged main`	Merged branches
`git merge --no-ff <branch>`	Merge with merge commit
`git merge --ff-only <branch>`	Only fast-forward
`git merge --squash <branch>`	Squash into staged changes
`git merge --abort`	Abort merge

9.3.5 Remotes

Command	Purpose
`git remote -v`	List remotes
`git remote add upstream <url>`	Add remote
`git fetch --all --prune`	Update all remotes
`git pull --rebase`	Fetch and rebase
`git push -u origin HEAD`	Push current branch, set upstream
`git push --force-with-lease --force-if-includes`	Safe force push
`git push origin --delete <branch>`	Delete remote branch
`git fetch origin pull/123/head:pr-123`	Fetch a GitHub PR

9.3.6 Rewrite and undo

Command	Purpose
`git rebase main`	Replay branch on main
`git rebase -i --autosquash main`	Interactive cleanup
`git rebase --onto <new> <old> <branch>`	Transplant commits
`git rebase --update-refs main`	Rebase a branch stack
`git rebase --continue / --abort`	Continue / abort
`git cherry-pick -x <sha>`	Backport a commit
`git reset --soft HEAD~N`	Uncommit, keep staged
`git reset HEAD~1`	Uncommit, keep unstaged
`git reset --hard <rev>`	Reset everything to rev

Command	Purpose
`git revert <sha>`	Undo with new commit
`git revert -m 1 <merge>`	Undo a merge
`git reflog`	History of HEAD moves
`git reset --hard ORIG_HEAD`	Undo last merge/rebase/reset
`git fsck --lost-found`	Find dangling objects

9.3.7 Stash and tags

Command	Purpose
`git stash push -u -m "msg"`	Stash including untracked
`git stash list` / `show -p`	Inspect stashes
`git stash pop` / `apply stash@{n}`	Restore
`git stash branch <name>`	Branch from stash
`git tag -a v1.2.0 -m "msg"`	Annotated tag
`git tag -s v1.2.0 -m "msg"`	Signed tag
`git push origin v1.2.0` / `--follow-tags`	Push tags
`git describe --tags --always --dirty`	Build version string

9.3.8 Power tools

Command	Purpose
`git bisect start <bad> <good>`	Start bisect
`git bisect run <script>`	Automated bisect
`git worktree add ../dir -b <branch>`	Extra working directory
`git sparse-checkout set <dirs>`	Limit checkout to dirs
`git submodule update --init --recursive`	Init submodules
`git subtree add --prefix=dir <url> main --squash`	Vendor a repo
`git lfs track "*.bin"`	Track with LFS
`git maintenance start`	Background optimization
`git cat-file -p <obj>` / `-t`	Inspect object / type
`git rev-parse HEAD` / `--abbrev-ref HEAD`	Hash / branch name

Command	Purpose
`git count-objects -vH`	Repo size

9.3.9 Security

Command	Purpose
`git config --global gpg.format ssh`	Use SSH signing
`git config --global commit.gpgsign true`	Sign all commits
`git log --show-signature -1`	Verify signature
`git verify-tag v1.2.0`	Verify tag
`gitleaks git -v`	Scan history for secrets
`pre-commit run --all-files`	Run all hooks
`git filter-repo --invert-paths --path <file>`	Purge file from history
`git filter-repo --replace-text repl.txt`	Redact strings in history
`git config --global --add safe.directory <path>`	Trust a repo path (CI/containers)

9.4 Key Takeaways

- A good Git workflow is a written set of decisions (branching, merge method, commit format, versioning, protection, deployment, secrets), each justified by how the team releases.
- Guardrails belong at every layer: local hooks for speed, CI and server rules for enforcement.
- Traceability from commit to PR to tag to artifact to running workload is the core value Git brings to DevOps.
- Fix on main first, backport with `cherry-pick -x`, release with immutable signed tags.
- In interviews, explain the why: three trees, object model, merge vs rebase trade-offs, reset vs revert, and incident response for leaks and force pushes.
- Keep the cheat sheet close, but understand what each command does to the working directory, index, HEAD and refs. That understanding is what turns a Git user into the engineer the team calls when things break.